

Temporal Planning Semantics

Mohammad Abdulaziz Lukas Koller

This is a formalisation of the semantics of temporal planning as used in the Planning Domain Definition Language. Based on that, we also develop a plan validator which is verified w.r.t. those semantics. It is based on an earlier formalisation of the semantics of classical planning [2]. This work was described in an earlier publication [1].

Contents

1	Temporal PDDL Semantics	4
1.1	Utility Functions	4
1.2	Abstract Syntax	5
1.2.1	Generic Entities	5
1.2.2	Domains	5
1.2.3	Problems	6
1.2.4	Plans	7
1.2.5	Ground Actions	7
1.3	Closed-World Assumption, Equality, and Negation	7
1.3.1	Proper Generalization	9
1.4	Happening Execution Semantics	9
1.5	Well-Formedness of PDDL	11
1.6	PDDL Semantics	15
1.7	Preservation of Well-Formedness	22
1.7.1	Well-Formed Action Instances	22
1.7.2	Preservation	25
1.7.3	Completeness of Semantics	27
2	Executable Temporal PDDL Checker	32
2.1	Generic DFS Reachability Checker	33
2.2	Implementation Refinements	34
2.2.1	Of-Type	34
2.2.2	Ground Action Interference & Application of Effects	38
2.2.3	Well-Formedness	38
2.2.4	Happening Execution	40
2.2.5	Construction of Induced Happening Sequence	41
2.3	Executable Plan Checker	59

2.4	Code Setup	61
2.4.1	Code Equations	61
2.4.2	Setup for Containers Framework	63
2.4.3	More Efficient Distinctness Check for Linorders	63
2.4.4	Parsing Rational Numbers	63
2.4.5	Code Generation	65
3	Connecting the Semantics to State-Transition Based Semantics	66
3.1	State-Transition Based Semantics	67
3.2	Connecting the State-Transition Based Semantics to Happening Sequence Based Semantics	69

```

theory Error-Monad-Add
imports
  Certification-Monads.Check-Monad
  Show.Show-Instances
begin

  abbreviation assert-opt  $\Phi \equiv$  if  $\Phi$  then Some () else None

  definition lift-opt  $m\ e \equiv$  case  $m$  of Some  $x \Rightarrow$  Error-Monad.return  $x$  | None  $\Rightarrow$ 
    Error-Monad.error  $e$ 

  lemma lift-opt-simps[simp]:
    lift-opt None  $e =$  error  $e$ 
    lift-opt (Some  $v$ )  $e =$  return  $v$ 
    <proof>

  lemma reflcl-image-iff[simp]:  $R^= "S = S \cup R" S$  <proof>

  named-theorems return-iff

  lemma bind-return-iff[return-iff]: Error-Monad.bind  $m\ f =$  Inr  $y \longleftrightarrow (\exists x. m$ 
     $=$  Inr  $x \wedge f\ x =$  Inr  $y)$ 
    <proof>

  lemma lift-opt-return-iff[return-iff]: lift-opt  $m\ e =$  Inr  $x \longleftrightarrow m =$  Some  $x$ 
    <proof>

  lemma check-return-iff[return-iff]: check  $\Phi\ e =$  Inr  $uu \longleftrightarrow \Phi$ 
    <proof>

  lemma check-simps[simp]:
    check True  $e =$  succeed
    check False  $e =$  error  $e$ 
    <proof>

  lemma Let-return-iff[return-iff]: (let  $x=v$  in  $f\ x$ )  $=$  Inr  $w \longleftrightarrow f\ v =$  Inr  $w$ 
    <proof>

  abbreviation ERR :: shows  $\Rightarrow$  (unit  $\Rightarrow$  shows) where ERR  $s \equiv \lambda-. s$ 
  abbreviation ERRS :: string  $\Rightarrow$  (unit  $\Rightarrow$  shows) where ERRS  $s \equiv$  ERR (shows
     $s$ )

  lemma error-monad-bind-split:  $P$  (bind  $m\ f$ )  $\longleftrightarrow (\forall v. m =$  Inl  $v \longrightarrow P$  (Inl  $v))$ 
     $\wedge (\forall v. m =$  Inr  $v \longrightarrow P$  ( $f\ v$ ))
    <proof>

```

lemma *error-monad-bind-split-asm*: $P \text{ (bind } m \text{ } f) \longleftrightarrow \neg (\exists x. m = \text{Inl } x \wedge \neg P$
 $(\text{Inl } x) \vee (\exists x. m = \text{Inr } x \wedge \neg P (f x)))$
 $\langle \text{proof} \rangle$

lemmas *error-monad-bind-splits = error-monad-bind-split error-monad-bind-split-asm*

end

1 Temporal PDDL Semantics

theory *TEMPORAL-PDDL-Semantics*

imports

Propositional-Proof-Systems.Formulas
Propositional-Proof-Systems.Sema
Propositional-Proof-Systems.Consistency
Automatic-Refinement.Misc
Automatic-Refinement.Refine-Util
HOL.Rat
Error-Monad-Add

begin

no-notation *insert* $(- \triangleright - [56,55] \ 55)$

1.1 Utility Functions

abbreviation *strict-sorted::'a::{\linorder} list $\Rightarrow$ bool* **where** *strict-sorted* $\equiv$ *sorted-wrt*
 $(<)$

definition *index-by* $f \ l \equiv \text{map-of } (\text{map } (\lambda x. (f \ x, x)) \ l)$

lemma *index-by-eq-Some-eq[simp]*:

assumes *distinct* $(\text{map } f \ l)$

shows $\text{index-by } f \ l \ n = \text{Some } x \longleftrightarrow (x \in \text{set } l \wedge f \ x = n)$

$\langle \text{proof} \rangle$

lemma *index-by-eq-SomeD*:

shows $\text{index-by } f \ l \ n = \text{Some } x \implies (x \in \text{set } l \wedge f \ x = n)$

$\langle \text{proof} \rangle$

lemma *lookup-zip-idx-eq*:

assumes $\text{length } \text{params} = \text{length } \text{args}$

assumes $i < \text{length } \text{args}$

assumes *distinct* params

assumes $k = \text{params} ! i$

shows $\text{map-of } (\text{zip } \text{params } \text{args}) \ k = \text{Some } (\text{args} ! i)$

$\langle \text{proof} \rangle$

lemma *rtrancl-image-idem[simp]*: $R^* \text{ `` } R^* \text{ `` } s = R^* \text{ `` } s$
<proof>

1.2 Abstract Syntax

1.2.1 Generic Entities

Defines the time number type

type-synonym *time* = *rat*

type-synonym *name* = *String.literal*

datatype *predicate* = *Pred* (*name*: *name*)

datatype *func* = *Func* (*name*: *name*)

Some of the AST entities are defined over a polymorphic *'val* type, which gets either instantiated by variables (for domains) or objects (for problems).

An atom is either a predicate with arguments, or an equality statement.

datatype *'ent atom* = *predAtm* (*predicate*: *predicate*) (*arguments*: *'ent list*)
 | *eqAtm* (*lhs*: *'ent*) (*rhs*: *'ent*)

A type is a list of primitive type names. To model a primitive type, we use a singleton list.

datatype *type* = *Either* (*primitives*: *name list*)

An effect contains a list of values to be added, and a list of values to be removed.

datatype *'ent ast-effect* = *Effect* (*adds*: (*'ent atom formula*) *list*)
 (*dels*: (*'ent atom formula*) *list*)

Variables are identified by their names.

datatype *variable* = *Var* (*varname*: *name*)

Objects and constants are identified by their names

datatype *object* = *Obj* (*name*: *name*) | *FuncEnt* (*func*: *func*) (*arguments*: *object list*) | *TimeEnt* *time*

datatype *term* = *VAR* *variable* | *CONST* *object*

hide-const (**open**) *VAR CONST* — Refer to constructors by qualified names only

1.2.2 Domains

A time specifier specifies the time point of conditions and effects in a durative action

datatype *temporal-annotation* = *At-Start* | *Over-All* | *At-End*

datatype *duration-op* = *EQ* | *LEQ* | *GEQ*

datatype *'ent duration-constraint* =
No-Const
 | *Time-Const* (*d-op*: *duration-op*) *time*
 | *Func-Const* (*d-op*: *duration-op*) (*func*: *func*) (*arguments*: *'ent list*)

An action schema has a name, a typed parameter list, a precondition, and an effect. In case of a durative action schema there is additionally a duration constraint, and the conditions and effects are timed with a time specifier

datatype *ast-action-schema* =
Simple-Action-Schema
 (*name*: *name*)
 (*parameters*: (*variable* $\times$ *type*) *list*)
 (*precondition*: *term atom formula*)
 (*effect*: *term ast-effect*)
 | *Durative-Action-Schema*
 (*name*: *name*)
 (*parameters*: (*variable* $\times$ *type*) *list*)
 (*duration-constraint*: *term duration-constraint list*)
 (*condition*: (*temporal-annotation* $\times$ (*term atom formula*)) *list*)
 (*durative-effect*: (*temporal-annotation* $\times$ (*term ast-effect*)) *list*)

A predicate declaration contains the predicate's name and its argument types.

datatype *predicate-decl* = *PredDecl* (*pred*: *predicate*) (*argTs*: *type list*)

datatype *function-decl* = *FuncDecl* (*func*: *func*) (*argTs*: *type list*)

A domain contains the declarations of primitive types, predicates, and action schemas.

datatype *ast-domain* = *Domain*
 (*types*: (*name* $\times$ *name*) *list*) — (*type*, *supertype*) declarations.
 (*predicates*: *predicate-decl list*)
 (*functions*: *function-decl list*)
 (*consts*: (*object* $\times$ *type*) *list*)
 (*actions*: *ast-action-schema list*)

1.2.3 Problems

A fact is a predicate applied to objects.

type-synonym *fact* = *predicate* $\times$ *object list*

A problem consists of a domain, a list of objects, a description of the initial state, and a description of the goal state.

datatype *ast-problem* = *Problem*
 (*domain*: *ast-domain*)
 (*objects*: (*object* $\times$ *type*) *list*)
 (*init*: *object atom formula list*)
 (*goal*: *object atom formula*)

1.2.4 Plans

datatype *plan-action* =
Simple-Plan-Action (*name*: *name*) (*arguments*: *object list*)
 | *Durative-Plan-Action* (*name*: *name*) (*arguments*: *object list*) (*duration*: *time*)

type-synonym *plan* = (*time* $\times$ *plan-action*) *list*

1.2.5 Ground Actions

The following datatype represents an action scheme that has been instantiated by replacing the arguments with concrete objects, also called ground action.

datatype *ground-action* = *Ground-Action*
 (*ga-name*: *name*)
 (*timing*: *temporal-annotation*)
 (*precondition*: (*object atom*) *formula*)
 (*effect*: *object ast-effect*)

A happening represents a collection of grounded actions that are all executed simultaneously at a specified time point.

type-synonym *happening* = *time* $\times$ (*ground-action list*)

1.3 Closed-World Assumption, Equality, and Negation

Discriminator for atomic predicate formulas.

fun *is-predAtom* **where**
is-predAtom (*Atom* (*predAtm* - -)) = *True* | *is-predAtom* - = *False*

fun *is-eqAtom* **where**
is-eqAtom (*Atom* (*eqAtm* - -)) = *True* | *is-eqAtom* - = *False*

The world model is a set of (atomic) formulas

type-synonym *world-model* = *object atom formula set*

It is basic, if it only contains atoms

definition *wm-basic* *M* $\equiv \forall a \in M. \text{is-predAtom } a$

A valuation extracted from the atoms of the world model

definition *valuation* :: *world-model* $\Rightarrow$ *object atom valuation*

where *valuation* $M \equiv \lambda predAtm\ p\ xs \Rightarrow Atom\ (predAtm\ p\ xs) \in M \mid eqAtm\ a\ b \Rightarrow a=b$

Augment a world model by adding negated versions of all atoms not contained in it, as well as interpretations of equality.

definition *close-world* :: *world-model* $\Rightarrow$ *world-model* **where**

close-world $M =$

$M \cup \{\neg(Atom\ (predAtm\ p\ as)) \mid p\ as.\ Atom\ (predAtm\ p\ as) \notin M\}$
 $\cup \{Atom\ (eqAtm\ a\ a) \mid a.\ True\} \cup \{\neg(Atom\ (eqAtm\ a\ b)) \mid a\ b.\ a \neq b\}$

definition *close-neg* $M \equiv M \cup \{\neg(Atom\ a) \mid a.\ Atom\ a \notin M\}$

lemma *wm-basic* $M \Longrightarrow close_world\ M = close_neg\ (M \cup \{Atom\ (eqAtm\ a\ a) \mid a.\ True\})$

<proof>

abbreviation *cw-entailment* (**infix** $^c \models =$ 53) **where**

$M\ ^c \models =\ \varphi \equiv close_world\ M\ \models\ \varphi$

lemma

close-world-extensive: $M \subseteq close_world\ M$ **and**

close-world-idem[simp]: $close_world\ (close_world\ M) = close_world\ M$

<proof>

lemma *in-close-world-conv*:

$\varphi \in close_world\ M \longleftrightarrow ($

$\varphi \in M$

$\vee (\exists p\ as.\ \varphi = \neg(Atom\ (predAtm\ p\ as)) \wedge Atom\ (predAtm\ p\ as) \notin M)$

$\vee (\exists a.\ \varphi = Atom\ (eqAtm\ a\ a))$

$\vee (\exists a\ b.\ \varphi = \neg(Atom\ (eqAtm\ a\ b)) \wedge a \neq b)$

$)$

<proof>

lemma *valuation-aux-1*:

fixes $M :: world_model$ **and** $\varphi :: object\ atom\ formula$

defines $C \equiv close_world\ M$

assumes $A: \forall \varphi \in C.\ \mathcal{A} \models \varphi$

shows $\mathcal{A} = valuation\ M$

<proof>

lemma *valuation-aux-2*:

assumes *wm-basic* M

shows $(\forall G \in close_world\ M.\ valuation\ M \models G)$

<proof>

lemma *val-imp-close-world*: $valuation\ M \models \varphi \Longrightarrow M\ ^c \models =\ \varphi$

<proof>

lemma *close-world-imp-val*:

wm-basic $M \Longrightarrow M\ ^c \models =\ \varphi \Longrightarrow valuation\ M \models \varphi$

$\langle proof \rangle$

Main theorem of this section: If a world model M contains only atoms, its induced valuation satisfies a formula φ if and only if the closure of M entails φ .

Note that there are no syntactic restrictions on φ , in particular, φ may contain negation.

theorem *valuation-iff-close-world:*

assumes *wm-basic* M

shows *valuation* $M \models \varphi \longleftrightarrow M^c \models \varphi$

$\langle proof \rangle$

1.3.1 Proper Generalization

Adding negation and equality is a proper generalization of the case without negation and equality

fun *is-STRIPS-fmla* :: 'ent atom formula $\Rightarrow$ bool **where**

is-STRIPS-fmla (*Atom* (*predAtm* -)) $\longleftrightarrow$ *True*

| *is-STRIPS-fmla* ($\perp$) $\longleftrightarrow$ *True*

| *is-STRIPS-fmla* ($\varphi_1 \wedge \varphi_2$) $\longleftrightarrow$ *is-STRIPS-fmla* $\varphi_1 \wedge$ *is-STRIPS-fmla* φ_2

| *is-STRIPS-fmla* ($\varphi_1 \vee \varphi_2$) $\longleftrightarrow$ *is-STRIPS-fmla* $\varphi_1 \wedge$ *is-STRIPS-fmla* φ_2

| *is-STRIPS-fmla* ($\neg \perp$) $\longleftrightarrow$ *True*

| *is-STRIPS-fmla* - $\longleftrightarrow$ *False*

lemma *aux1*: $\llbracket \text{wm-basic } M; \text{is-STRIPS-fmla } \varphi; \text{valuation } M \models \varphi; \forall G \in M. \mathcal{A} \models G \rrbracket \Longrightarrow \mathcal{A} \models \varphi$

$\langle proof \rangle$

lemma *aux2*: $\llbracket \text{wm-basic } M; \text{is-STRIPS-fmla } \varphi; \forall \mathcal{A}. (\forall G \in M. \mathcal{A} \models G) \longrightarrow \mathcal{A} \models \varphi \rrbracket \Longrightarrow \text{valuation } M \models \varphi$

$\langle proof \rangle$

lemma *valuation-iff-STRIPS:*

assumes *wm-basic* M

assumes *is-STRIPS-fmla* φ

shows *valuation* $M \models \varphi \longleftrightarrow M \models \varphi$

$\langle proof \rangle$

Our extension to negation and equality is a proper generalization of the standard STRIPS semantics for formula without negation and equality

theorem *proper-STRIPS-generalization:*

$\llbracket \text{wm-basic } M; \text{is-STRIPS-fmla } \varphi \rrbracket \Longrightarrow M^c \models \varphi \longleftrightarrow M \models \varphi$

$\langle proof \rangle$

1.4 Happening Execution Semantics

For this section, we fix a domain D , using Isabelle's locale mechanism.

```

locale ast-domain =
  fixes D :: ast-domain
begin

```

Check if two ground actions are interfering. This definition is taken from [FL03]

```

definition acts-non-intrf :: ground-action  $\Rightarrow$  ground-action  $\Rightarrow$  bool where
  acts-non-intrf a b  $\longleftrightarrow$  (
    let
      adda = set(adds(effect a)); dela = set(dels(effect a)); prea = Atom ' atoms
      (precondition a);
      addb = set(adds(effect b)); delb = set(dels(effect b)); preb = Atom ' atoms
      (precondition b)
    in
      prea  $\cap$  (addb  $\cup$  delb) = {}  $\wedge$ 
      preb  $\cap$  (adda  $\cup$  dela) = {}  $\wedge$ 
      adda  $\cap$  delb = {}  $\wedge$ 
      addb  $\cap$  dela = {}
  )

```

lemma *acts-non-intrf-symmetric*:

```

 $\forall a\ b. \text{acts-non-intrf } a\ b \longleftrightarrow \text{acts-non-intrf } b\ a$ 
<proof>

```

It seems to be agreed upon that, in case of a contradictory effect, addition overrides deletion. We model this behaviour by first executing the deletions, and then the additions.

```

fun apply-happ :: happening  $\Rightarrow$  world-model  $\Rightarrow$  world-model where
  apply-happ (ti, Ai) M =
    (M -  $\bigcup$  (set (map (set o dels o effect) Ai)))  $\cup$   $\bigcup$  (set (map (set o adds o effect) Ai))

```

Predicate to model that the given list happenings is executable, and transforms an initial world model *M* into a final model *M'*.

Note that this definition over the list structure is more convenient in HOL than to explicitly define an indexed sequence $M_0 \dots M_N$ of intermediate world models, as done in [Lif87].

```

fun valid-happ-seq :: world-model  $\Rightarrow$  happening list  $\Rightarrow$  world-model  $\Rightarrow$  bool where
  valid-happ-seq M [] M'  $\longleftrightarrow$  (M = M')
| valid-happ-seq M ((ti, Ai) # hs) M'  $\longleftrightarrow$ 
  ( $\forall a \in \text{set } A_i. M \stackrel{c}{\models} \text{precondition } a$ )
   $\wedge$  (pairwise acts-non-intrf (set Ai))
   $\wedge$  valid-happ-seq (apply-happ (ti, Ai) M) hs M'

```

lemma *valid-happ-seq-M_j-is-M_i*:

assumes $\forall h \in \text{set } hs. \text{apply-happ } h\ M_i = M_i$

and *valid-happ-seq* *M_i* *hs* *M_j*

shows *M_i* = *M_j*

<proof>

end — Context of *ast-domain*

1.5 Well-Formedness of PDDL

fun *ty-term* **where**

ty-term *varT* *objT* (*term.VAR* *v*) = *varT* *v*
| *ty-term* *varT* *objT* (*term.CONST* *c*) = *objT* *c*

lemma *ty-term-mono*: *varT* $\subseteq_m$ *varT'* $\implies$ *objT* $\subseteq_m$ *objT'* $\implies$
ty-term *varT* *objT* $\subseteq_m$ *ty-term* *varT'* *objT'*
<proof>

context *ast-domain* **begin**

The signature is a partial function that maps the predicates of the domain to lists of argument types.

definition *sig* :: *predicate* $\rightarrow$ *type list* **where**

sig $\equiv$ *map-of* (*map* (λ *PredDecl* *p* *n* $\Rightarrow$ (*p,n*)) (*predicates* *D*))

definition *func-sig* :: *func* $\rightarrow$ *type list* **where**

func-sig $\equiv$ *map-of* (*map* (λ *FuncDecl* *f* *n* $\Rightarrow$ (*f,n*)) (*functions* *D*))

We use a flat subtype hierarchy, where every type is a subtype of object, and there are no other subtype relations.

Note that we do not need to restrict this relation to declared types, as we will explicitly ensure that all types used in the problem are declared.

fun *subtype-edge* **where**

subtype-edge (*ty*, *superty*) = (*superty*, *ty*)

definition *subtype-rel* $\equiv$ *set* (*map* *subtype-edge* (*types* *D*))

definition *of-type* :: *type* $\Rightarrow$ *type* $\Rightarrow$ *bool* **where**

of-type *oT* *T* $\equiv$ *set* (*primitives* *oT*) $\subseteq$ *subtype-rel** “ *set* (*primitives* *T*)

This checks that every primitive on the LHS is contained in or a subtype of a primitive on the RHS

For the next few definitions, we fix a partial function that maps a polymorphic entity type '*e* to types. An entity can be instantiated by variables or objects later.

context

fixes *ty-ent* :: '*ent* $\rightarrow$ *type* — Entity's type, None if invalid
begin

Checks whether an entity has a given type

definition *is-of-type* :: 'ent $\Rightarrow$ type $\Rightarrow$ bool **where**
is-of-type v T $\longleftrightarrow$ (
 case ty-ent v of
 Some vT $\Rightarrow$ of-type vT T
 | None $\Rightarrow$ False)

fun *wf-pred-atom* :: predicate $\times$ 'ent list $\Rightarrow$ bool **where**
wf-pred-atom (p,vs) $\longleftrightarrow$ (
 case sig p of
 None $\Rightarrow$ False
 | Some Ts $\Rightarrow$ list-all2 *is-of-type* vs Ts)

fun *wf-func-args* :: func $\times$ 'ent list $\Rightarrow$ bool **where**
wf-func-args (f,vs) $\longleftrightarrow$ (
 case func-sig f of
 None $\Rightarrow$ False
 | Some Ts $\Rightarrow$ list-all2 *is-of-type* vs Ts)

Predicate-atoms are well-formed if their arguments match the signature, equalities are well-formed if the arguments are valid objects (have a type).

TODO: We could check that types may actually overlap

fun *wf-atom* :: 'ent atom $\Rightarrow$ bool **where**
wf-atom (predAtm p vs) $\longleftrightarrow$ *wf-pred-atom* (p,vs)
 | *wf-atom* (eqAtm a b) $\longleftrightarrow$ ty-ent a $\neq$ None $\wedge$ ty-ent b $\neq$ None

A formula is well-formed if it consists of valid atoms, and does not contain negations, except for the encoding $\neg \perp$ of true.

fun *wf-fmla* :: ('ent atom) formula $\Rightarrow$ bool **where**
wf-fmla (Atom a) $\longleftrightarrow$ *wf-atom* a
 | *wf-fmla* ($\perp$) $\longleftrightarrow$ True
 | *wf-fmla* ($\varphi 1 \wedge \varphi 2$) $\longleftrightarrow$ (*wf-fmla* $\varphi 1 \wedge$ *wf-fmla* $\varphi 2$)
 | *wf-fmla* ($\varphi 1 \vee \varphi 2$) $\longleftrightarrow$ (*wf-fmla* $\varphi 1 \wedge$ *wf-fmla* $\varphi 2$)
 | *wf-fmla* ($\neg \varphi$) $\longleftrightarrow$ *wf-fmla* φ
 | *wf-fmla* ($\varphi 1 \rightarrow \varphi 2$) $\longleftrightarrow$ (*wf-fmla* $\varphi 1 \wedge$ *wf-fmla* $\varphi 2$)

lemma *wf-fmla* $\varphi = (\forall a \in \text{atoms } \varphi. \text{wf-atom } a)$
 <proof>

Special case for a well-formed atomic predicate formula

fun *wf-fmla-atom* **where**
wf-fmla-atom (Atom (predAtm a vs)) $\longleftrightarrow$ *wf-pred-atom* (a,vs)
 | *wf-fmla-atom* - $\longleftrightarrow$ False

lemma *wf-fmla-atom-alt*: *wf-fmla-atom* $\varphi \longleftrightarrow$ *is-predAtom* $\varphi \wedge$ *wf-fmla* φ
 <proof>

An effect is well-formed if the added and removed formulas are atomic

fun *wf-effect* **where**

$wf_effect (Effect\ a\ d) \longleftrightarrow (\forall ae \in set\ a. wf_fmla_atom\ ae) \wedge (\forall de \in set\ d. wf_fmla_atom\ de)$

```
fun wf-duration-const :: 'ent duration-constraint  $\Rightarrow$  bool where
  wf-duration-const No-Const  $\longleftrightarrow$  True
| wf-duration-const (Time-Const op d)  $\longleftrightarrow$   $d \geq 0$ 
| wf-duration-const (Func-Const op f vs)  $\longleftrightarrow$ 
  (case func-sig f of
    None  $\Rightarrow$  False
  | Some Ts  $\Rightarrow$  list-all2 is-of-type vs Ts)
```

definition $wf_duration_consts = list_all\ wf_duration_const$
end — Context fixing ty_ent

definition $constT :: object \rightarrow type$ **where**
 $constT \equiv map_of\ (consts\ D)$

An action schema is well-formed if the parameter names are distinct, and the precondition and effect is well-formed wrt. the parameters.

```
fun wf-action-schema :: ast-action-schema  $\Rightarrow$  bool where
  wf-action-schema (Simple-Action-Schema n params pre eff)  $\longleftrightarrow$  (
    let tyt = ty-term (map-of params) constT in
    distinct (map fst params)
     $\wedge$  wf-fmla tyt pre
     $\wedge$  wf-effect tyt eff)
| wf-action-schema (Durative-Action-Schema n params d cond eff)  $\longleftrightarrow$  (
  let tyt = ty-term (map-of params) constT in
  distinct (map fst params)
   $\wedge$   $(\forall (t,c) \in set\ cond. wf\_fmla\ tyt\ c)$ 
   $\wedge$   $(\forall (t,e) \in set\ eff. wf\_effect\ tyt\ e \wedge t \neq Over-All)$ 
   $\wedge$  wf-duration-consts tyt d)
```

A type is well-formed if it consists only of declared primitive types, and the type object.

```
fun wf-type where
  wf-type (Either Ts)  $\longleftrightarrow$  set Ts  $\subseteq$  insert (STR "object") (fst'set (types D))
```

A predicate is well-formed if its argument types are well-formed.

```
fun wf-predicate-decl where
  wf-predicate-decl (PredDecl p Ts)  $\longleftrightarrow$   $(\forall T \in set\ Ts. wf\_type\ T)$ 
```

A function is well-formed if its argument types are well-formed.

```
fun wf-function-decl where
  wf-function-decl (FuncDecl f Ts)  $\longleftrightarrow$   $(\forall T \in set\ Ts. wf\_type\ T)$ 
```

The types declaration is well-formed, if all supertypes are declared types (or object)

definition $wf\text{-}types \equiv snd'set (types D) \subseteq insert (STR "object") (fst'set (types D))$

A domain is well-formed if

- there are no duplicate declared predicate names,
- all declared predicates are well-formed,
- there are no duplicate declared function names,
- all declared functions are well-formed,
- there are no duplicate action names,
- and all declared actions are well-formed

definition $wf\text{-}domain :: bool$ **where**

$wf\text{-}domain \equiv$
 $wf\text{-}types$
 $\wedge distinct (map (predicate\text{-}decl.pred) (predicates D))$
 $\wedge (\forall p \in set (predicates D). wf\text{-}predicate\text{-}decl p)$
 $\wedge distinct (map (function\text{-}decl.func) (functions D))$
 $\wedge (\forall f \in set (functions D). wf\text{-}function\text{-}decl f)$
 $\wedge distinct (map fst (consts D))$
 $\wedge (\forall (n, T) \in set (consts D). wf\text{-}type T)$
 $\wedge distinct (map ast\text{-}action\text{-}schema.name (actions D))$
 $\wedge (\forall a \in set (actions D). wf\text{-}action\text{-}schema a)$

end — locale $ast\text{-}domain$

We fix a problem, and also include the definitions for the domain of this problem.

locale $ast\text{-}problem = ast\text{-}domain$ $domain P$
for $P :: ast\text{-}problem$
begin

We refer to the problem domain as D

abbreviation $D \equiv ast\text{-}problem.domain P$

definition $objT :: object \rightarrow type$ **where**
 $objT \equiv map\text{-}of (objects P) ++ constT$

lemma $objT\text{-}alt$: $objT = map\text{-}of (consts D @ objects P)$
 $\langle proof \rangle$

definition $wf\text{-}fact :: fact \Rightarrow bool$ **where**
 $wf\text{-}fact = wf\text{-}pred\text{-}atom objT$

fun *wf-func-assign* :: *object atom formula* $\Rightarrow$ *bool* **where**
wf-func-assign (*Atom* (*eqAtm* (*FuncEnt* *f args*) (*TimeEnt* *d*))) $\longleftrightarrow$ *wf-func-args*
objT (*f, args*)
| *wf-func-assign* - $\longleftrightarrow$ *False*

lemma *wf-func-assign-is-eqAtom*: *wf-func-assign f* $\implies$ *is-eqAtom f*
 $\langle proof \rangle$

This definition is needed for well-formedness of the initial model, and forward-references to the concept of world model.

definition *wf-world-model* **where**
wf-world-model *M* = ($\forall f \in M. \text{wf-fmla-atom } objT f$)

definition *wf-problem* **where**
wf-problem $\equiv$
wf-domain
 $\wedge distinct (\text{map } fst (\text{objects } P) @ \text{map } fst (\text{consts } D))$
 $\wedge (\forall (n, T) \in set (\text{objects } P). \text{wf-type } T)$
 $\wedge distinct (\text{init } P)$
 $\wedge (\forall f \in set (\text{init } P). \text{wf-fmla-atom } objT f \vee \text{wf-func-assign } f)$
 $\wedge \text{wf-fmla } objT (\text{goal } P)$

fun *wf-effect-inst* :: *object ast-effect* $\Rightarrow$ *bool* **where**
wf-effect-inst (*Effect* (*a*) (*d*)) $\longleftrightarrow$ ($\forall a \in set a \cup set d. \text{wf-fmla-atom } objT a$)

lemma *wf-effect-inst-alt*: *wf-effect-inst eff* = *wf-effect objT eff*
 $\langle proof \rangle$

end — locale *ast-problem*

Locale to express a well-formed domain

locale *wf-ast-domain* = *ast-domain* +
assumes *wf-domain*: *wf-domain*

Locale to express a well-formed problem

locale *wf-ast-problem* = *ast-problem P* **for** *P* +
assumes *wf-problem*: *wf-problem*
begin
sublocale *wf-ast-domain domain P*
 $\langle proof \rangle$

end — locale *wf-ast-problem*

1.6 PDDL Semantics

context *ast-domain* **begin**

definition *resolve-action-schema* :: *name* $\rightarrow$ *ast-action-schema* **where**
resolve-action-schema *n* = *index-by ast-action-schema.name (actions D) n*

fun *subst-term* **where**
subst-term psubst (term.VAR x) = psubst x
| subst-term psubst (term.CONST c) = c

To instantiate an action schema, we first compute a substitution from parameters to objects, and then apply this substitution to the precondition and effect. The substitution is applied via the *map-xxx* functions generated by the datatype package.

fun *instantiate-action-schema* :: *ast-action-schema* $\Rightarrow$ *object list* $\Rightarrow$ *temporal-annotation* $\Rightarrow$ *ground-action* **where**
instantiate-action-schema (Simple-Action-Schema n params pre eff) args ta =
(let
tsubst = subst-term (the o (map-of (zip (map fst params) args)));
pre-inst = (map-formula o map-atom) tsubst pre;
eff-inst = (map-ast-effect) tsubst eff
in
Ground-Action n ta pre-inst eff-inst
)

Filter a given timed-list by a time specifier. (**at start ...**), (**at end ...**), or (**over all ...**) conditions/effects

definition *filter-time-spec* :: *temporal-annotation* $\Rightarrow$ (*temporal-annotation* $\times$ 'a) *list* $\Rightarrow$ 'a *list* **where**
filter-time-spec t l = map snd (filter (((=) t) o fst) l)

lemma *filter-time-spec-correct*: $\forall x \in \text{set } (\text{filter-time-spec } t \text{ } xs). (t, x) \in \text{set } xs$
<proof>

auxiliary-function to conjunct multiple effects into one single effect. Needed to construct effect after filtering by a time specifier

definition *conjunct-effects* :: ('a *ast-effect*) *list* $\Rightarrow$ 'a *ast-effect* ($\bigwedge_{eff} - [40] \ 40$) **where**
conjunct-effects l = Effect ((concat o (map adds)) l) ((concat o (map dels)) l)

instantiate a given Durative Action Schema; Note: if an action schema is well formed then the effect for the snap action for the temporal annotation (**over all ...**) will be an empty effect (**Effect [] []**)

fun *inst-snap-action* :: *ast-action-schema* $\Rightarrow$ *object list* $\Rightarrow$ *temporal-annotation* $\Rightarrow$ *ground-action* **where**
inst-snap-action (Durative-Action-Schema n params d cond eff) args ta =
(let
tsubst = subst-term (the o (map-of (zip (map fst params) args)));
pre_i = (map-formula o map-atom) tsubst (BigAnd (filter-time-spec ta cond));
eff_i = (map-ast-effect) tsubst ($\bigwedge_{eff}$ (filter-time-spec ta eff))

```

    in
      Ground-Action n ta prei effi
    )

```

end — Context of *ast-domain*

context *wf-ast-domain* **begin**

Resolving an action yields a well-founded action schema.

```

lemma resolve-action-wf:
  assumes resolve-action-schema n = Some a
  shows wf-action-schema a
  ⟨proof⟩

```

end — Context of *ast-domain*

context *ast-problem* **begin**

Initial model

```

definition I :: world-model where
  I ≡ set (filter is-predAtom (init P))

```

Resolve a plan action and instantiate the referenced action schema.

```

fun res-inst :: plan-action ⇒ temporal-annotation ⇒ ground-action option where
  res-inst (Simple-Plan-Action n args) ta =
    Some (instantiate-action-schema (the (resolve-action-schema n)) args ta)
  | res-inst (Durative-Plan-Action - -) - = None

```

```

fun res-inst-snap-action :: plan-action ⇒ temporal-annotation → ground-action
where
  res-inst-snap-action (Durative-Plan-Action n args d) ta =
    Some (inst-snap-action (the (resolve-action-schema n)) args ta)
  | res-inst-snap-action (Simple-Plan-Action - -) - = None

```

Check whether object has specified type

```

definition is-obj-of-type n T ≡ case objT n of
  None ⇒ False
  | Some oT ⇒ of-type oT T

```

We can also use the generic *is-of-type* function.

```

lemma is-obj-of-type-alt: is-obj-of-type = is-of-type objT
  ⟨proof⟩

```

HOL encoding of matching an action's formal parameters against an argument list. The parameters of the action are encoded as a list of *name*×*type* pairs, such that we map it to a list of types first. Then, the list relator

list-all2 checks that arguments and types have the same length, and each matching pair of argument and type satisfies the predicate *is-obj-of-type*.

definition *action-params-match* *a args*
 $\equiv \text{list-all2 } \text{is-obj-of-type } \text{args } (\text{map } \text{snd } (\text{parameters } a))$

All functions are required to be constant and defined in the initial state. Therefore all duration constraints must be satisfied by the initial state.

definition *func-eval* :: *object* $\rightarrow$ *rat* **where**
func-eval $\equiv \text{map-of } (\text{fold } (\lambda a \text{ fev. case } a \text{ of } \text{Atom } (\text{eqAtm } f (\text{TimeEnt } t)) \Rightarrow (f,t)\#\text{fev} \mid - \Rightarrow \text{fev}) (\text{init } P) [])$

fun *duration-matches* :: *time* $\Rightarrow$ *term duration-constraint* $\Rightarrow$ (*variable* $\times$ *type*)
list $\Rightarrow$ *object list* $\Rightarrow$ *bool* **where**
duration-matches *d No-Const params args* $\longleftrightarrow d \geq 0$
 $\mid \text{duration-matches } d (\text{Time-Const op } d') \text{ params args} \longleftrightarrow$
 $(\text{case op of}$
 $\quad \text{LEQ} \Rightarrow d \leq d'$
 $\quad \mid \text{EQ} \Rightarrow d = d'$
 $\quad \mid \text{GEQ} \Rightarrow d \geq d')$
 $\mid \text{duration-matches } d (\text{Func-Const op } f \text{ vs}) \text{ params args} \longleftrightarrow$
 $(\text{let } tsubst = \text{subst-term } (\text{the } o (\text{map-of } (\text{zip } (\text{map } \text{fst } \text{params}) \text{ args}))) \text{ in}$
 $\text{case func-eval } (\text{FuncEnt } f (\text{map } tsubst \text{ vs})) \text{ of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad \mid \text{Some } d' \Rightarrow (\text{case op of}$
 $\quad \quad \text{LEQ} \Rightarrow d \leq d'$
 $\quad \quad \mid \text{EQ} \Rightarrow d = d'$
 $\quad \quad \mid \text{GEQ} \Rightarrow d \geq d'))$

definition *durations-match* *d xs params args* = *list-all* ($\lambda x. \text{duration-matches } d \text{ } x \text{ params args}$) *xs*

At this point, we can define well-formedness of a plan action: The action must refer to a declared action schema, the arguments must be compatible with the formal parameters' types.

fun *wf-plan-action* :: *plan-action* $\Rightarrow$ *bool* **where**
wf-plan-action (*Simple-Plan-Action* *n args*) = (
 $\text{case resolve-action-schema } n \text{ of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad \mid \text{Some } (\text{Simple-Action-Schema } n \text{ params pre eff}) \Rightarrow$
 $\quad \quad \text{action-params-match } (\text{Simple-Action-Schema } n \text{ params pre eff}) \text{ args}$
 $\quad \mid \text{Some } (\text{Durative-Action-Schema } n \text{ params d cond eff}) \Rightarrow \text{False})$
 $\mid \text{wf-plan-action } (\text{Durative-Plan-Action } n \text{ args } d) =$
 $\quad \text{case resolve-action-schema } n \text{ of}$
 $\quad \quad \text{None} \Rightarrow \text{False}$
 $\quad \quad \mid \text{Some } (\text{Simple-Action-Schema } n \text{ params pre eff}) \Rightarrow \text{False}$
 $\quad \quad \mid \text{Some } (\text{Durative-Action-Schema } n \text{ params dconst cond eff}) \Rightarrow$
 $\quad \quad \quad \text{action-params-match } (\text{Durative-Action-Schema } n \text{ params dconst cond eff})$
args

$$\wedge \text{durations-match } d \text{ dconst params args } \wedge d \geq 0)$$

A plan is wellformed if all plan actions are wellformed and all starting time points are greater-equal 0.

definition *wf-plan* :: *plan* $\Rightarrow$ *bool* **where**
wf-plan $\pi s \longleftrightarrow (\forall (t, \pi) \in \text{set } \pi s. \text{wf-plan-action } \pi \wedge t \geq 0)$

definition *is-act-simple* :: *plan-action* $\Rightarrow$ *bool* **where**
is-act-simple $\pi \longleftrightarrow (\text{case } \pi \text{ of } (\text{Simple-Plan-Action } -) \Rightarrow \text{True} \mid - \Rightarrow \text{False})$

definition *simple-acts* :: *plan* $\Rightarrow$ (*time* $\times$ *plan-action*) *set* **where**
simple-acts $\pi s = \text{set } (\text{filter } (\text{is-act-simple } o \text{snd}) \pi s)$

lemma *simple-acts-subset*: *simple-acts* $\pi s \subseteq \text{set } \pi s$
 $\langle \text{proof} \rangle$

definition *durative-acts* :: *plan* $\Rightarrow$ (*time* $\times$ *plan-action*) *set* **where**
durative-acts $\pi s = \text{set } (\text{filter } ((\text{HOL.Not}) o \text{is-act-simple } o \text{snd}) \pi s)$

lemma *dur-acts-subset*: *durative-acts* $\pi s \subseteq \text{set } \pi s$
 $\langle \text{proof} \rangle$

lemma *is-act-simple-alt*: *is-act-simple* = *is-Simple-Plan-Action*
 $\langle \text{proof} \rangle$

Definition of a happening time point. Happening time point specify time points at, which the current state can change. Therefore every start and end of any plan action is a happening time point.

definition *is-htp* :: *plan* $\Rightarrow$ *time* $\Rightarrow$ *bool* **where**
is-htp $\pi s \ t_i \longleftrightarrow (\exists \pi. (t_i, \pi) \in \text{set } \pi s) \vee (\exists (t_\pi, \pi) \in \text{durative-acts } \pi s. t_i = t_\pi + (\text{duration } \pi))$

Predicate for sequence of happening time points.

definition *htps-seq* :: *plan* $\Rightarrow$ *time list* $\Rightarrow$ *bool* **where**
htps-seq $\pi s \ \text{htps} \longleftrightarrow \text{strict-sorted } \text{htps} \wedge (\forall t. t \in \text{set } \text{htps} \longleftrightarrow \text{is-htp } \pi s \ t)$

Predicate for consecutive happening time points: There is no happening time point in between.

definition *consec-htps* :: *plan* $\Rightarrow$ *time* $\Rightarrow$ *time* $\Rightarrow$ *bool* **where**
consec-htps $\pi s \ t_i \ t_j \longleftrightarrow (t_i < t_j \wedge \text{is-htp } \pi s \ t_i \wedge \text{is-htp } \pi s \ t_j \wedge (\forall t. \text{is-htp } \pi s \ t \longrightarrow t \leq t_i \vee t_j \leq t))$

Predicate to check is a grounded action is a instantiation of a plan action for an induced happening sequence.

fun *inst-of-plan-action* :: *plan* $\Rightarrow$ *time* $\times$ *plan-action* $\Rightarrow$ *time* $\times$ *ground-action* $\Rightarrow$ *bool* **where**

$$\begin{aligned}
& \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_a, a) \longleftrightarrow (\\
& \quad (\text{res-inst } \pi \text{ At-Start} = \text{Some } a \wedge t_\pi = t_a) \\
& \quad \vee (\text{res-inst-snap-action } \pi \text{ At-Start} = \text{Some } a \wedge t_\pi = t_a) \\
& \quad \vee (\text{res-inst-snap-action } \pi \text{ At-End} = \text{Some } a \wedge t_\pi + (\text{duration } \pi) = t_a) \\
& \quad \vee (\text{res-inst-snap-action } \pi \text{ Over-All} = \text{Some } a \wedge \\
& \quad (\exists t_i t_j. \text{consec-htps } \pi s t_i t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi) \wedge t_i < t_a \\
& \quad \wedge t_a < t_j)))
\end{aligned}$$

abbreviation *happ-member* (**infix** $\in_h$ 53) **where**

$$ta \in_h hs \equiv (\text{case } ta \text{ of } (t, a) \Rightarrow (\exists as. (t, as) \in \text{set } hs \wedge a \in \text{set } as))$$

Definition of an induced happening sequence.

- for every simple action (t, π) there is an instantiation at time point t
- for every durative action (t, π)
 - there is a ground action at time point t , which contains all conditions and effect of π with time specifier (**at start** ...)
 - there is a ground action at time point $t + (\text{duration } \pi)$, which contains all conditions and effect of π with time specifier (**at end** ...)
 - in between every consecutive happening-time-points $t_i t_j$ during the execution of π there is a ground action at some time point t' (in between t_i & t_j), which contains all conditions of π with time specifier (**over all** ...), the effect of is empty
- additionally an induced happening sequence is strictly sorted and contains no other actions besides the ones specified above.

definition *ind-happ-seq* :: *plan* $\Rightarrow$ *happening list* $\Rightarrow$ *bool* **where**

$$\begin{aligned}
& \text{ind-happ-seq } \pi s hs \longleftrightarrow (\\
& \quad \text{strict-sorted } (\text{map } \text{fst } hs) \\
& \quad \wedge (\forall (t_\pi, \pi) \in \text{simple-acts } \pi s. \\
& \quad \quad \text{let } \\
& \quad \quad \quad \pi_{start} = \text{the } (\text{res-inst } \pi \text{ At-Start}) \\
& \quad \quad \text{in} \\
& \quad \quad \quad \exists A. (t_\pi, A) \in \text{set } hs \wedge \pi_{start} \in \text{set } A) \\
& \quad \wedge (\forall (t_\pi, \pi) \in \text{durative-acts } \pi s. \\
& \quad \quad \text{let } \pi_{start} = \text{the } (\text{res-inst-snap-action } \pi \text{ At-Start}); \\
& \quad \quad \quad \pi_{end} = \text{the } (\text{res-inst-snap-action } \pi \text{ At-End}); \\
& \quad \quad \quad \pi_{inv} = \text{the } (\text{res-inst-snap-action } \pi \text{ Over-All}) \text{ in} \\
& \quad \quad (\exists A. (t_\pi, A) \in \text{set } hs \wedge \pi_{start} \in \text{set } A) \wedge \\
& \quad \quad (\exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge \pi_{end} \in \text{set } A) \wedge
\end{aligned}$$

$$\begin{aligned}
& (\forall t_i t_j. (\text{consec-htps } \pi s t_i t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + (\text{duration } \pi)) \\
& \quad \longrightarrow (\exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge \pi_{inv} \in \text{set } A))) \\
& \wedge (\forall (t_i, A_i) \in \text{set } hs. A_i \neq \square \wedge \\
& \quad (\forall a \in \text{set } A_i. \exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_i, a))))
\end{aligned}$$

lemma *ind-happ-seq-props:*

assumes *ind-happ-seq* πs hs

shows *strict-sorted* (*map fst* hs)

$$\begin{aligned}
& (t_\pi, \pi) \in \text{simple-acts } \pi s \implies \exists A. (t_\pi, A) \in \text{set } hs \wedge \text{the } (\text{res-inst } \pi \text{ At-Start}) \\
& \in \text{set } A \\
& (t_\pi, \pi) \in \text{durative-acts } \pi s \implies \exists A. (t_\pi, A) \in \text{set } hs \wedge \text{the } (\text{res-inst-snap-action } \\
& \pi \text{ At-Start}) \in \text{set } A \\
& (t_\pi, \pi) \in \text{durative-acts } \pi s \implies \exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge \text{the } \\
& (\text{res-inst-snap-action } \pi \text{ At-End}) \in \text{set } A \\
& \llbracket (t_\pi, \pi) \in \text{durative-acts } \pi s; \text{consec-htps } \pi s t_i t_j; t_\pi \leq t_i; t_j \leq t_\pi + (\text{duration } \\
& \pi) \rrbracket \\
& \implies \exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge \text{the } (\text{res-inst-snap-action } \pi \\
& \text{Over-All}) \in \text{set } A \\
& (t_i, A_i) \in \text{set } hs \implies A_i \neq \square \\
& \llbracket (t_i, A_i) \in \text{set } hs; a \in \text{set } A_i \rrbracket \implies \exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s \\
& (t_\pi, \pi) (t_i, a) \\
& \langle \text{proof} \rangle
\end{aligned}$$

A sequence of plan actions form a path, if they are well-formed and their instantiations form a path.

definition *plan-happ-path* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**
plan-happ-path $M \pi s M' \longleftrightarrow (\exists hs. \text{ind-happ-seq } \pi s hs \wedge \text{valid-happ-seq } M hs M')$

A plan is valid wrt. a given initial model, if it forms a path to a goal model

definition *valid-plan-from2* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *bool* **where**
valid-plan-from2 $M \pi s \longleftrightarrow (\text{wf-plan } \pi s \wedge (\exists M'. \text{plan-happ-path } M \pi s M' \wedge M' \models (\text{goal } P)))$

Finally, a plan is valid if it is valid wrt. the initial world model I

definition *valid-plan2* :: *plan* $\Rightarrow$ *bool*
where *valid-plan2* $\equiv \text{valid-plan-from2 } I$

Concise definition used in paper:

lemma *valid-plan2* $\pi s \equiv \text{wf-plan } \pi s \wedge (\exists M'. \text{plan-happ-path } I \pi s M' \wedge M' \models (\text{goal } P))$
 $\langle \text{proof} \rangle$

end — Context of *ast-problem*

1.7 Preservation of Well-Formedness

1.7.1 Well-Formed Action Instances

The goal of this section is to establish that well-formedness of world models is preserved by execution of well-formed plan actions.

context *ast-problem* **begin**

As plan actions are executed by first instantiating them, and then executing the action instance, it is natural to define a well-formedness concept for action instances.

fun *wf-ground-action* :: *ground-action* $\Rightarrow$ *bool* **where**
wf-ground-action (*Ground-Action* - - *pre eff*) $\longleftrightarrow$ (*wf-fmla* *objT pre* $\wedge$ *wf-effect* *objT eff*)

We first prove that instantiating a well-formed action schema will yield a well-formed action instance.

We begin with some auxiliary lemmas before the actual theorem.

lemma (**in** *ast-domain*) *of-type-refl*[*simp*, *intro!*]: *of-type* *T* *T*
 $\langle proof \rangle$

lemma (**in** *ast-domain*) *of-type-trans*[*trans*]:
of-type *T1* *T2* $\Longrightarrow$ *of-type* *T2* *T3* $\Longrightarrow$ *of-type* *T1* *T3*
 $\langle proof \rangle$

lemma *is-of-type-map-ofE*:
assumes *is-of-type* (*map-of* *params*) *x* *T*
obtains *i* *xT* **where** *i* < *length* *params* *params*!*i* = (*x*, *xT*) *of-type* *xT* *T*
 $\langle proof \rangle$

lemma *wf-atom-mono*:
assumes *SS*: *tys* $\subseteq_m$ *tys'*
assumes *WF*: *wf-atom* *tys* *a*
shows *wf-atom* *tys'* *a*
 $\langle proof \rangle$

lemma *wf-fmla-atom-mono*:
assumes *SS*: *tys* $\subseteq_m$ *tys'*
assumes *WF*: *wf-fmla-atom* *tys* *a*
shows *wf-fmla-atom* *tys'* *a*
 $\langle proof \rangle$

lemma *constT-ss-objT*: *constT* $\subseteq_m$ *objT*
 $\langle proof \rangle$

lemma *wf-atom-constT-imp-objT*: *wf-atom* (*ty-term* *Q* *constT*) *a* $\Longrightarrow$ *wf-atom* (*ty-term* *Q* *objT*) *a*

$\langle proof \rangle$

lemma *wf-fmla-atom-constT-imp-objT*: *wf-fmla-atom* (ty-term *Q* const*T*) *a* $\implies$
wf-fmla-atom (ty-term *Q* obj*T*) *a*
 $\langle proof \rangle$

context

fixes *Q* and *f* :: *variable* $\Rightarrow$ *object*

assumes *INST*: *is-of-type* *Q* *x* *T* $\implies$ *is-of-type* obj*T* (*f* *x*) *T*

begin

lemma *is-of-type-var-conv*: *is-of-type* (ty-term *Q* obj*T*) (term.VAR *x*) *T* $\longleftrightarrow$
is-of-type *Q* *x* *T*
 $\langle proof \rangle$

lemma *is-of-type-const-conv*: *is-of-type* (ty-term *Q* obj*T*) (term.CONST *x*) *T*
 $\longleftrightarrow$ *is-of-type* obj*T* *x* *T*
 $\langle proof \rangle$

lemma *INST'*: *is-of-type* (ty-term *Q* obj*T*) *x* *T* $\implies$ *is-of-type* obj*T* (subst-term
f *x*) *T*
 $\langle proof \rangle$

lemma *wf-inst-eq-aux*: *Q* *x* = *Some* *T* $\implies$ obj*T* (*f* *x*) $\neq$ *None*
 $\langle proof \rangle$

lemma *wf-inst-eq-aux'*: ty-term *Q* obj*T* *x* = *Some* *T* $\implies$ obj*T* (subst-term *f* *x*)
 $\neq$ *None*
 $\langle proof \rangle$

lemma *wf-inst-atom*:

assumes *wf-atom* (ty-term *Q* const*T*) *a*

shows *wf-atom* obj*T* (map-atom (subst-term *f*) *a*)

$\langle proof \rangle$

lemma *wf-inst-formula-atom*:

assumes *wf-fmla-atom* (ty-term *Q* const*T*) *a*

shows *wf-fmla-atom* obj*T* ((map-formula o map-atom o subst-term) *f* *a*)

$\langle proof \rangle$

lemma *wf-inst-effect*:

assumes *wf-effect* (ty-term *Q* const*T*) φ

shows *wf-effect* obj*T* ((map-ast-effect o subst-term) *f* φ)

$\langle proof \rangle$

lemma *wf-inst-formula*:

assumes *wf-fmla* (ty-term *Q* const*T*) φ

shows *wf-fmla objT* ((*map-formula o map-atom o subst-term*) *f* φ)
 ⟨*proof*⟩

lemma *wf-BigAnd-aux*: $\forall x \in \text{set } xs. \text{wf-fmla } tyt \ x \implies \text{wf-fmla } tyt \ (\text{BigAnd } xs)$
 ⟨*proof*⟩

lemma *wf-BigAnd-filter-time-spec*:
fixes *n params d cond eff*
defines *a* $\equiv$ *Durative-Action-Schema* *n params d cond eff*
assumes *wf-action-schema a*
shows *wf-fmla* (*ty-term* (*map-of params*) *constT*) (*BigAnd* (*filter-time-spec ta cond*))
 ⟨*proof*⟩

lemma *wf-conjunct-effects-aux*: $\forall x \in \text{set } xs. \text{wf-effect } tyt \ x \implies \text{wf-effect } tyt \ (\bigwedge_{eff} xs)$
 ⟨*proof*⟩

lemma *wf-conj-eff-filter-time-spec*:
fixes *n params d cond eff*
defines *a* $\equiv$ *Durative-Action-Schema* *n params d cond eff*
assumes *wf-action-schema a*
shows *wf-effect* (*ty-term* (*map-of params*) *constT*) ($\bigwedge_{eff}$ (*filter-time-spec ta eff*))
 ⟨*proof*⟩

end

Next two theorems show that, instantiating a well-formed action schema with compatible arguments will yield a well-formed action instance.

theorem *wf-inst-action-schema*:
fixes *n params pre eff*
defines *a* $\equiv$ *Simple-Action-Schema* *n params pre eff*
assumes *action-params-match a args*
assumes *wf-action-schema a*
shows *wf-ground-action* (*instantiate-action-schema a args ta*)
 ⟨*proof*⟩

theorem *wf-inst-durative-action-schema*:
fixes *n params d cond eff*
defines *a* $\equiv$ *Durative-Action-Schema* *n params d cond eff*
assumes *action-params-match a args*
assumes *wf-action-schema a*
shows *wf-ground-action* (*inst-snap-action a args ta*)
 ⟨*proof*⟩

lemma *wf-over-all-empty-eff*:
assumes *wf-plan-action* π
and *wf-action-schema* *a_{schema}*

and *Some* $a_{schema} = resolve-action-schema (name \pi)$
and *Some* $a = res-inst-snap-action \pi \text{ Over-All}$
shows $effect\ a = Effect\ []\ []$
 $\langle proof \rangle$

end — Context of *ast-problem*

1.7.2 Preservation

context *ast-problem* **begin**

fun *happ-precond* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**
 $happ-precond\ (t, A)\ M \longleftrightarrow (\forall a \in set\ A.\ M \models^c precondition\ a)$

fun *happ-non-intrf* :: *happening* $\Rightarrow$ *bool* **where**
 $happ-non-intrf\ (t, A) \longleftrightarrow (\forall a \in set\ A.\ \forall b \in set\ A.\ a \neq b \longrightarrow acts-non-intrf\ a\ b)$

Shorthand for enabled happening: all preconditions satisfied, no interfering actions, & all ground actions are wellformed .

fun *happ-enabled* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**
 $happ-enabled\ (t, A)\ M \longleftrightarrow (happ-precond\ (t, A)\ M \wedge happ-non-intrf\ (t, A) \wedge (\forall a \in set\ A.\ wf-ground-action\ a))$

Shorthand for wellformed-ness of a happening sequence: all ground actions wellformed .

fun *wf-happ-seq* :: *happening list* $\Rightarrow$ *bool* **where**
 $wf-happ-seq\ hs \longleftrightarrow (\forall (t, A) \in set\ hs.\ (\forall a \in set\ A.\ wf-ground-action\ a))$

end — Context of *ast-problem*

context *wf-ast-problem* **begin**

The initial world model is well-formed

lemma *wf-I*: *wf-world-model* *I*
 $\langle proof \rangle$

lemma *wf-ground-action-wf-fmla-atom*:
 $wf-ground-action\ a \implies \forall x \in set\ ((adds\ o\ effect)\ a).\ wf-fmla-atom\ objT\ x$
 $\langle proof \rangle$

The Application of a happening preserves well-formedness

theorem *wf-apply-happ*:
assumes *happ-enabled* $(t, as)\ s$
and *wf-world-model* s
shows *wf-world-model* $(apply-happ\ (t, as)\ s)$
 $\langle proof \rangle$

theorem *wf-apply-happ-compact-notation:*
 $\text{happ-enabled } (t, as) \ s \implies \text{wf-world-model } s \implies \text{wf-world-model } (\text{apply-happ } (t, as) \ s)$
 $\langle \text{proof} \rangle$

corollary *wf-valid-happ-seq:*
assumes *wf-world-model* M
and *wf-happ-seq* hs
and *valid-happ-seq* $M \ hs \ M'$
shows *wf-world-model* M'
 $\langle \text{proof} \rangle$

In a wellformed plan the action parameters match for every plan action

lemma *wf-plan-action-params-match:*
assumes *wf-plan* πs
and $(t, a) \in \text{set } \pi s$
and $a_{\text{schema}} = \text{the } (\text{resolve-action-schema } (\text{name } a))$
shows *action-params-match* $a_{\text{schema}} \ (\text{arguments } a)$
 $\langle \text{proof} \rangle$

lemma *simple-plan-action-schema-type1:*
assumes *wf-plan-action* $(\text{Simple-Plan-Action } n \ \text{args})$
shows $\exists \text{ params pre eff. } \text{resolve-action-schema } n = \text{Some } (\text{Simple-Action-Schema } n \ \text{params pre eff})$
 $\langle \text{proof} \rangle$

lemma *durative-plan-action-schema-type1:*
assumes *wf-plan-action* $(\text{Durative-Plan-Action } n \ \text{args } t')$
shows $\exists \text{ params } d \ \text{cond eff. } \text{resolve-action-schema } n = \text{Some } (\text{Durative-Action-Schema } n \ \text{params } d \ \text{cond eff})$
 $\langle \text{proof} \rangle$

Instantiating a wellformed plan action always produces a wellformed ground action

lemma *wf-inst-of-plan-action:*
assumes *wf-plan* πs
and $(t_\pi, \pi) \in \text{set } \pi s$
and *inst-of-plan-action* $\pi s \ (t_\pi, \pi) \ (t_i, a)$
shows *wf-ground-action* a
 $\langle \text{proof} \rangle$

Every ground action in an induced happening sequence for a wellformed plan is wellformed.

lemma *happ-seq-wf-ground-action:*
assumes *wf-plan* πs
and *ind-happ-seq* $\pi s \ hs$
and $(t_i, A_i) \in \text{set } hs$

and $a \in \text{set } A_i$
shows *wf-ground-action* a
 $\langle \text{proof} \rangle$

An induced happening sequence for a wellformed plan is always a wellformed happening sequence.

lemma *inst-wf-plan-wf-happ-seq*:
assumes *wf-plan* πs
and *ind-happ-seq* πs hs
shows *wf-happ-seq* hs
 $\langle \text{proof} \rangle$

Execution of a plan preserves well-formedness

corollary *wf-plan-happ-path*:
assumes *wf-world-model* M **and** *wf-plan* πs **and** *plan-happ-path* M πs M'
shows *wf-world-model* M'
 $\langle \text{proof} \rangle$

end — Context of *wf-ast-problem*

Auxiliary function that are used to divide happening sequences.

definition $leq \equiv \lambda t_i. \lambda(t_a, as). t_a \leq t_i$

definition $le \equiv \lambda t_i. \lambda(t_a, as). t_a < t_i$

lemma *strict-sorted-app-iff*: *strict-sorted* $a \wedge \text{strict-sorted } b \wedge (\forall x \in \text{set } a. \forall y \in \text{set } b. x < y)$
 $\longleftrightarrow \text{strict-sorted } (a @ b)$
 $\langle \text{proof} \rangle$

lemma *empty-filter-conv*: $\llbracket \] = \text{filter } P \ xs \longleftrightarrow (\forall x \in \text{set } xs. \neg P \ x)$
 $\langle \text{proof} \rangle$

1.7.3 Completeness of Semantics

context *wf-ast-problem*
begin

fun *consume2* **where**
consume2 $\llbracket \] = 0$
 $\mid \text{consume2 } (x \# xs) (y \# ys) = 0 + \text{consume2 } xs \ ys$

lemma *empty-happ*:
 $(\bigwedge a. a \in \text{set } A \implies \text{effect } a = \text{Effect } \llbracket \]) \implies \text{apply-happ } (t, A) \ M = M$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-1*:
 $\llbracket \bigwedge t \ A \ a. \llbracket (t, A) \in \text{set } hs1; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } \llbracket \] ;$
 $\text{valid-happ-seq } M \ hs2 \ M'$

$(\bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs1}; a \in \text{set } A \rrbracket \implies M^c \models \text{precondition } a);$
 $(\bigwedge t A a. (t, A) \in \text{set } \text{hs1} \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies$
 $\text{valid-happ-seq } M (\text{hs1} @ \text{hs2}) M'$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-2:*

$\llbracket \text{valid-happ-seq } M (\text{hs1} @ \text{hs2}) M'; \bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs1}; a \in \text{set } A \rrbracket \implies$
 $\text{effect } a = \text{Effect } [] [] \rrbracket \implies$
 $\text{valid-happ-seq } M \text{hs2 } M'$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-2':*

$\llbracket \text{valid-happ-seq } M (\text{hs1} @ \text{hs2}) M'; \bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs2}; a \in \text{set } A \rrbracket \implies$
 $\text{effect } a = \text{Effect } [] [] \rrbracket \implies$
 $\text{valid-happ-seq } M \text{hs1 } M'$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-3:*

$\llbracket (t, A) \in \text{set } \text{hs1}; a \in \text{set } A; \text{valid-happ-seq } M (\text{hs1} @ \text{hs2}) M';$
 $\bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs1}; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] [] \rrbracket$
 $\implies M^c \models \text{precondition } a$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-4:*

$\llbracket \text{valid-happ-seq } M (\text{hs1} @ \text{hs2}) M'; \bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs1}; a \in \text{set } A \rrbracket \implies \text{effect}$
 $a = \text{Effect } [] [] \rrbracket \implies$
 $(\bigwedge t A a. (t, A) \in \text{set } \text{hs1} \implies \text{pairwise acts-non-intrf } (\text{set } A))$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-empty-eff:*

$\llbracket \bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs}; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] [];$
 $(\bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs}; a \in \text{set } A \rrbracket \implies M^c \models \text{precondition } a);$
 $(\bigwedge t A a. (t, A) \in \text{set } \text{hs} \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies$
 $\text{valid-happ-seq } M \text{hs } M$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-append-5:*

$\llbracket \bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs2}; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } [] [];$
 $\text{valid-happ-seq } M \text{hs1 } M';$
 $(\bigwedge t A a. \llbracket (t, A) \in \text{set } \text{hs2}; a \in \text{set } A \rrbracket \implies M'^c \models \text{precondition } a);$
 $(\bigwedge t A a. (t, A) \in \text{set } \text{hs2} \implies \text{pairwise acts-non-intrf } (\text{set } A)) \rrbracket \implies$
 $\text{valid-happ-seq } M (\text{hs1} @ \text{hs2}) M'$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-cons:* $\llbracket \text{strict-sorted } (\text{map } f (x \# xs)); x' \in \text{set } xs \rrbracket \implies f x \neq f x'$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-order:*

$$\llbracket (\bigwedge x. x \in s \implies \exists y \in \text{set } (x1 \# xs1 @ x2 \# xs2). f y = x); \text{strict-sorted } (\text{map } f (x1 \# xs1 @ x2 \# xs2)) \rrbracket$$

$$(\bigwedge y. y \in \text{set } xs1 \implies f y \notin s); x \in s; x < f x2 \rrbracket \implies x = f x1$$

$$\langle \text{proof} \rangle$$

lemma *in-suff*:

$$\llbracket \bigwedge x. x \in \text{set } xs1 \implies x \neq x2; x2 \in \text{set } (xs1 @ xs2) \rrbracket \implies x2 \in \text{set } xs2$$

$$\langle \text{proof} \rangle$$

lemma *strict-sorted-order-in-suff*:

$$\llbracket \text{strict-sorted } (xs1 @ xs2); x1 \in \text{set } xs1; x2 \in \text{set } xs2 \rrbracket \implies x1 < x2$$

$$\langle \text{proof} \rangle$$

lemma *empty-eff-non-intrf*:

$$\llbracket \bigwedge a. a \in A \implies \text{effect } a = \text{Effect } [] \rrbracket \implies \text{pairwise acts-non-intrf } A$$

$$\langle \text{proof} \rangle$$

lemma *filter-eq-consD*: $x \# l' = \text{filter } Q l \implies x \in \text{set } l \wedge Q x$

$\langle \text{proof} \rangle$

lemma *valid-happ-seq-construct*:

assumes $\bigwedge t. t \in \text{htps} \implies \exists A. (t, A) \in \text{set } hs$
and $\bigwedge t. t \in \text{htps} \implies \exists A. (t, A) \in \text{set } hs'$
and $\text{strict-sorted } (\text{map } \text{fst } hs)$
and $\text{strict-sorted } (\text{map } \text{fst } hs')$
and $\bigwedge t A A'. \llbracket t \in \text{htps}; (t, A) \in \text{set } hs; (t, A') \in \text{set } hs' \rrbracket \implies \text{set } A = \text{set } A'$
and $\bigwedge t A A' a. \llbracket t \notin \text{htps}; (t, A) \in \text{set } hs; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } []$
and $\bigwedge t A A' a. \llbracket t \notin \text{htps}; (t, A) \in \text{set } hs'; a \in \text{set } A \rrbracket \implies \text{effect } a = \text{Effect } []$
and $\bigwedge t A t_1 t_2 a.$

$$\llbracket t \notin \text{htps}; t_1 \in \text{htps}; t_2 \in \text{htps}; t \in \{t_1 <..< t_2\}; (t, A) \in \text{set } hs; a \in \text{set } A \rrbracket \implies$$

$$\exists t' \in \{t_1 <..< t_2\}. \exists A'. (t', A') \in \text{set } hs' \wedge a \in \text{set } A'$$

and $\bigwedge t A t_1 t_2 a.$

$$\llbracket t \notin \text{htps}; t_1 \in \text{htps}; t_2 \in \text{htps}; t \in \{t_1 <..< t_2\}; (t, A) \in \text{set } hs'; a \in \text{set } A \rrbracket \implies$$

$$\exists t' \in \{t_1 <..< t_2\}. \exists A'. (t', A') \in \text{set } hs \wedge a \in \text{set } A'$$

and $\text{valid-happ-seq } M hs M'$
and $hs \neq [] \implies (\text{fst } (\text{hd } hs)) \in \text{htps}$
and $hs' \neq [] \implies (\text{fst } (\text{hd } hs')) \in \text{htps}$
and $hs \neq [] \implies (\text{fst } (\text{last } hs)) \in \text{htps}$
and $hs' \neq [] \implies (\text{fst } (\text{last } hs')) \in \text{htps}$
shows $\text{valid-happ-seq } M hs' M'$

$$\langle \text{proof} \rangle$$

lemma *not-htp-empty-effect*:

assumes $\text{wf-plan } \pi s$
and $\neg \text{is-htp } \pi s t_i$
and $\text{ind-happ-seq } \pi s hs$

and $(t_i, A_i) \in \text{set } hs$
and $a \in \text{set } A_i$
shows $(\text{effect } a) = \text{Effect } [] []$
 $\langle \text{proof} \rangle$

lemma *unique-happ*:

assumes *ind-happ-seq* πs hs
shows $\llbracket (t, A) \in \text{set } hs; (t, A') \in \text{set } hs \rrbracket \implies A = A'$
 $\langle \text{proof} \rangle$

lemma *inst-of-plan-actionE*:

$\bigwedge t_\pi \pi. \llbracket \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_a, a);$
 $\llbracket \text{res-inst } \pi \text{ At-Start} = \text{Some } a; t_\pi = t_a \rrbracket \implies Q;$
 $\llbracket \text{res-inst-snap-action } \pi \text{ At-Start} = \text{Some } a; t_\pi = t_a \rrbracket \implies Q;$
 $\llbracket \text{res-inst-snap-action } \pi \text{ At-End} = \text{Some } a; t_\pi + (\text{duration } \pi) = t_a \rrbracket \implies Q;$
 $\bigwedge t_i t_j. \llbracket \text{res-inst-snap-action } \pi \text{ Over-All} = \text{Some } a; \text{consec-htps } \pi s t_i t_j;$
 $t_\pi \leq t_i; t_j \leq t_\pi + (\text{duration } \pi); t_i < t_a; t_a < t_j \rrbracket \implies Q$
 $\implies Q$
 $\langle \text{proof} \rangle$

lemma *ind-happ-seq-happ-at-htps*:

assumes *is-htp* πs t
and *ind-happ-seq* πs hs
and *ind-happ-seq* πs hs'
and $(t, A) \in \text{set } hs$
and $(t, A') \in \text{set } hs'$
and $a \in \text{set } A$
shows $a \in \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *ind-happ-seq-happ-at-htps'*:

assumes *is-htp* πs t
and *ind-happ-seq* πs hs
and *ind-happ-seq* πs hs'
and $(t, A) \in \text{set } hs$
and $(t, A') \in \text{set } hs'$
shows $\text{set } A = \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *htp-in-ind-happ-seq*:

$\llbracket \text{is-htp } \pi s t; \text{ind-happ-seq } \pi s hs \rrbracket \implies \exists A. (t, A) \in \text{set } hs$
 $\langle \text{proof} \rangle$

lemma *sorted-takeWhile*: *strict-sorted* $xs \implies x \in \text{set } (\text{takeWhile } ((\lambda x. x < c)) xs)$
 $= (x \in \text{set } xs \wedge (x < c))$

$\langle \text{proof} \rangle$

lemma *sorted-dropWhile*: *strict-sorted* $xs \implies x \in \text{set } (\text{dropWhile } ((\lambda x. x \leq c)) xs)$
 $= (x \in \text{set } xs \wedge (c < x))$

$\langle proof \rangle$

lemma *map-fst-dropWhile*: $map\ fst\ (dropWhile\ (leq\ t)\ hs) = (dropWhile\ ((\lambda x. x \leq t))\ (map\ fst\ hs))$
 $\langle proof \rangle$

lemma *map-fst-takeWhile*: $map\ fst\ (takeWhile\ (le\ t)\ hs) = (takeWhile\ ((\lambda x. x < t))\ (map\ fst\ hs))$
 $\langle proof \rangle$

lemma *strict-sorted-fst-unique-snd*:
 $\llbracket strict-sorted\ (map\ fst\ hs); (t, A) \in set\ hs; (t, A') \in set\ hs \rrbracket \implies A = A'$
 $\langle proof \rangle$

lemma *strict-sorted-dropWhile*:
 $strict-sorted\ xs \implies strict-sorted\ (dropWhile\ Q\ xs)$
 $\langle proof \rangle$

lemma *strict-sorted-takeWhile*:
 $strict-sorted\ xs \implies strict-sorted\ (takeWhile\ Q\ xs)$
 $\langle proof \rangle$

lemma *strict-sorted-drop-leq-take-le'*:
assumes *strict-sorted* $(map\ fst\ hs)$
shows $(t_a, A) \in set\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (le\ t_j)\ hs)) \longleftrightarrow (t_i < t_a \wedge t_a < t_j \wedge (t_a, A) \in set\ hs) \text{ (is ?l } \longleftrightarrow ?r)$
 $\langle proof \rangle$

lemma *ind-happ-seqs-same-acts-leq_i-le_j'*:
assumes *is-htp* $\pi s\ t_i$
and *is-htp* $\pi s\ t_j$
and *ind-happ-seq* $\pi s\ hs_1$
and *ind-happ-seq* $\pi s\ hs_2$
and $(t_a, A) \in set\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (le\ t_j)\ hs_1))$
and $a \in set\ A$
shows $\exists t_a' A'. (t_a', A') \in set\ (dropWhile\ (leq\ t_i)\ (takeWhile\ (le\ t_j)\ hs_2)) \wedge a \in set\ A'$
 $\langle proof \rangle$

lemma *happs-same-between-htps*:
assumes *ind-happ-seq* $\pi s\ hs$
and *ind-happ-seq* $\pi s\ hs'$
and *is-htp* $\pi s\ t_1$ *is-htp* $\pi s\ t_2$
and $(t, A) \in set\ hs$
and $\neg is-htp\ \pi s\ t$
and $t_1 < t$
and $t < t_2$
and $a \in set\ A$
shows $\exists t' \in \{t_1 < .. < t_2\}. \exists A'. (t', A') \in set\ hs' \wedge a \in set\ A'$

<proof>

lemma *consec-https-props*:
 assumes *consec-https* πs t_i t_j
 shows $t_i < t_j$
 is-htp πs t_i
 is-htp πs t_j
 is-htp πs $t \implies t \leq t_i \vee t_j \leq t$
<proof>

lemma *ind-happ-seq-hd-htp*:
 assumes *ind-happ-seq* πs hs $hs \neq []$
 shows *is-htp* πs (*fst* (*hd* hs))
<proof>

lemma *sorted-last*: $\llbracket \text{strict-sorted } xs; x \in \text{set } xs \rrbracket \implies x \leq \text{last } xs$
<proof>

lemma *ind-happ-seq-last-htp*:
 assumes *ind-happ-seq* πs hs $hs \neq []$
 shows *is-htp* πs (*fst* (*last* hs))
<proof>

definition *https'* $\pi s \equiv \{t . \text{is-htp } \pi s \ t\}$

lemma *https-props*: $t \in \text{https}' \ \pi s \implies \text{is-htp } \pi s \ t$
<proof>

lemma *ind-happ-seq-unique-final-state'*:
 assumes *ind-happ-seq* πs hs **and** *ind-happ-seq* πs hs'
 shows $\llbracket \text{wf-plan } \pi s; \text{valid-happ-seq } M_0 \ hs \ M_n \rrbracket \implies \text{valid-happ-seq } M_0 \ hs' \ M_n$
<proof>

end

end — Theory

2 Executable Temporal PDDL Checker

theory *TEMPORAL-PDDL-Checker*
imports

TEMPORAL-PDDL-Semantics
 Error-Monad-Add
 HOL.String

HOL-Library.Code-Target-Nat
 HOL-Library.While-Combinator
 Containers.Containers

HOL—Library.Tree
begin

2.1 Generic DFS Reachability Checker

Used for subtype checks

definition *E-of-succ succ* $\equiv \{ (u,v). v \in \text{set } (\text{succ } u) \}$
lemma *succ-as-E*: $\text{set } (\text{succ } x) = E\text{-of-succ succ } \{x\}$
 $\langle \text{proof} \rangle$

context
fixes *succ* :: 'a $\Rightarrow$ 'a list
begin

private abbreviation (*input*) *E* $\equiv E\text{-of-succ succ}$

definition *dfs-reachable D w* $\equiv$
 $\text{let } (V, w, brk) = \text{while } (\lambda(V, w, brk). \neg brk \wedge w \neq []) (\lambda(V, w, -). \\$
 $\text{case } w \text{ of } v \# w \Rightarrow \\$
 $\text{if } D \ v \text{ then } (V, v \# w, \text{True}) \\$
 $\text{else if } v \in V \text{ then } (V, w, \text{False}) \\$
 $\text{else} \\$
 $\text{let } V = \text{insert } v \ V \text{ in} \\$
 $\text{let } w = \text{succ } v \ @ \ w \text{ in} \\$
 $(V, w, \text{False}) \\$
 $) (\{\}, w, \text{False}) \\$
 $\text{in } brk$

context
fixes *w₀* :: 'a list
assumes *finite-dfs-reachable[simp, intro!]*: *finite* (*E** “ *set w₀*)
begin

private abbreviation (*input*) *W₀* $\equiv \text{set } w_0$

definition *dfs-reachable-invar D V W brk* $\longleftrightarrow$
 $W_0 \subseteq W \cup V \\$
 $\wedge W \cup V \subseteq E^* \text{ “ } W_0 \\$
 $\wedge E \text{ “ } V \subseteq W \cup V \\$
 $\wedge \text{Collect } D \cap V = \{\} \\$
 $\wedge (brk \longrightarrow \text{Collect } D \cap E^* \text{ “ } W_0 \neq \{\})$

lemma *card-decreases*:
 $\llbracket \text{finite } V; y \notin V; \text{dfs-reachable-invar } D \ V \ (Set.\text{insert } y \ W) \ brk \rrbracket \\$
 $\implies \text{card } (E^* \text{ “ } W_0 - Set.\text{insert } y \ V) < \text{card } (E^* \text{ “ } W_0 - V) \\$
 $\langle \text{proof} \rangle$

lemma *all-neq-Cons-is-Nil*[simp]:

$(\forall y \text{ ys. } x2 \neq y \# \text{ys}) \longleftrightarrow x2 = [] \text{ } \langle \text{proof} \rangle$

lemma *dfs-reachable-correct*: $\text{dfs-reachable } D \text{ } w_0 \longleftrightarrow \text{Collect } D \cap E^* \text{ } \{\}$

$\langle \text{proof} \rangle$

end

definition *tab-succ* $l \equiv \text{Mapping.lookup-default } [] \text{ } (\text{fold } (\lambda(u,v). \text{Mapping.map-default } u \text{ } (\text{Cons } v)) \text{ } l \text{ } \text{Mapping.empty})$

lemma *Some-eq-map-option* [iff]: $(\text{Some } y = \text{map-option } f \text{ } x_0) = (\exists z. x_0 = \text{Some } z \wedge f \text{ } z = y)$

$\langle \text{proof} \rangle$

lemma *tab-succ-correct*: $E\text{-of-succ } (\text{tab-succ } l) = \text{set } l$

$\langle \text{proof} \rangle$

end

lemma *finite-imp-finite-dfs-reachable*:

$\llbracket \text{finite } E; \text{finite } S \rrbracket \implies \text{finite } (E^* \text{ } S)$

$\langle \text{proof} \rangle$

lemma *dfs-reachable-tab-succ-correct*: $\text{dfs-reachable } (\text{tab-succ } l) \text{ } D \text{ } vs_0 \longleftrightarrow \text{Collect } D \cap (\text{set } l)^* \text{ } \{\}$

$\langle \text{proof} \rangle$

2.2 Implementation Refinements

2.2.1 Of-Type

definition *of-type-impl* $G \text{ } oT \text{ } T \equiv (\forall pt \in \text{set } (\text{primitives } oT). \text{dfs-reachable } G \text{ } ((=) \text{ } pt) \text{ } (\text{primitives } T))$

fun *ty-term'* **where**

$\text{ty-term}' \text{ } varT \text{ } objT \text{ } (\text{term.VAR } v) = varT \text{ } v$

| $\text{ty-term}' \text{ } varT \text{ } objT \text{ } (\text{term.CONST } c) = \text{Mapping.lookup } objT \text{ } c$

lemma *ty-term'-correct-aux*: $\text{ty-term}' \text{ } varT \text{ } objT \text{ } t = \text{ty-term } varT \text{ } (\text{Mapping.lookup } objT \text{ } t)$

$\langle \text{proof} \rangle$

lemma *ty-term'-correct*[simp]: $\text{ty-term}' \text{ } varT \text{ } objT = \text{ty-term } varT \text{ } (\text{Mapping.lookup } objT)$

$\langle \text{proof} \rangle$

context *ast-domain* **begin**

definition *of-type1* $pt\ T \longleftrightarrow pt \in subtype-rel^* \text{ `` } set\ (primitives\ T)$

lemma *of-type-refine1*: *of-type* $oT\ T \longleftrightarrow (\forall pt \in set\ (primitives\ oT). \text{ of-type1 } pt\ T)$
 $\langle proof \rangle$

definition $STG \equiv (tab-succ\ (map\ subtype-edge\ (types\ D)))$

lemma *subtype-rel-impl*: $subtype-rel = E-of-succ\ (tab-succ\ (map\ subtype-edge\ (types\ D)))$
 $\langle proof \rangle$

lemma *of-type1-impl*: *of-type1* $pt\ T \longleftrightarrow dfs-reachable\ (tab-succ\ (map\ subtype-edge\ (types\ D)))\ ((=)pt)\ (primitives\ T)$
 $\langle proof \rangle$

lemma *of-type-impl-correct*: *of-type-impl* $STG\ oT\ T \longleftrightarrow of-type\ oT\ T$
 $\langle proof \rangle$

definition *mp-constT* :: (object, type) mapping **where**
 $mp-constT = Mapping.of-alist\ (consts\ D)$

lemma *mp-objT-correct[simp]*: $Mapping.lookup\ mp-constT = constT$
 $\langle proof \rangle$

Lifting the subtype-graph through wf-checker

context

fixes *ty-ent* :: 'ent $\rightarrow$ type — Entity's type, None if invalid
begin

definition *is-of-type'* $stg\ v\ T \longleftrightarrow$ (
 $case\ ty-ent\ v\ of$
 $\quad Some\ vT \Rightarrow of-type-impl\ stg\ vT\ T$
 $\quad | \ None \Rightarrow False$)

lemma *is-of-type'-correct*: *is-of-type'* $STG\ v\ T = is-of-type\ ty-ent\ v\ T$
 $\langle proof \rangle$

fun *wf-pred-atom'* **where** *wf-pred-atom'* $stg\ (p, vs) \longleftrightarrow$
 $(case\ sig\ p\ of$
 $\quad None \Rightarrow False$
 $\quad | \ Some\ Ts \Rightarrow list-all2\ (is-of-type'\ stg)\ vs\ Ts)$

lemma *wf-pred-atom'-correct*: *wf-pred-atom'* $STG\ pvs = wf-pred-atom\ ty-ent\ pvs$
 $\langle proof \rangle$

```

fun wf-func-args' where wf-func-args' stg (f,args)  $\longleftrightarrow$ 
  (case func-sig f of
    None  $\Rightarrow$  False
  | Some Ts  $\Rightarrow$  list-all2 (is-of-type' stg) args Ts)

```

lemma wf-func-args'-correct: wf-func-args' STG fargs = wf-func-args ty-ent fargs
 <proof>

```

fun wf-atom' :: -  $\Rightarrow$  'ent atom  $\Rightarrow$  bool where
  wf-atom' stg (atom.predAtm p vs)  $\longleftrightarrow$  wf-pred-atom' stg (p,vs)
| wf-atom' stg (atom.eqAtm a b) = (ty-ent a  $\neq$  None  $\wedge$  ty-ent b  $\neq$  None)

```

lemma wf-atom'-correct: wf-atom' STG a = wf-atom ty-ent a
 <proof>

```

fun wf-fmla' :: -  $\Rightarrow$  ('ent atom) formula  $\Rightarrow$  bool where
  wf-fmla' stg (Atom a)  $\longleftrightarrow$  wf-atom' stg a
| wf-fmla' stg  $\perp$   $\longleftrightarrow$  True
| wf-fmla' stg ( $\varphi 1 \wedge \varphi 2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi 1 \wedge$  wf-fmla' stg  $\varphi 2$ )
| wf-fmla' stg ( $\varphi 1 \vee \varphi 2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi 1 \wedge$  wf-fmla' stg  $\varphi 2$ )
| wf-fmla' stg ( $\varphi 1 \rightarrow \varphi 2$ )  $\longleftrightarrow$  (wf-fmla' stg  $\varphi 1 \wedge$  wf-fmla' stg  $\varphi 2$ )
| wf-fmla' stg ( $\neg \varphi$ )  $\longleftrightarrow$  wf-fmla' stg  $\varphi$ 

```

lemma wf-fmla'-correct: wf-fmla' STG φ $\longleftrightarrow$ wf-fmla ty-ent φ
 <proof>

```

fun wf-fmla-atom1' where
  wf-fmla-atom1' stg (Atom (predAtm p vs))  $\longleftrightarrow$  wf-pred-atom' stg (p,vs)
| wf-fmla-atom1' stg -  $\longleftrightarrow$  False

```

lemma wf-fmla-atom1'-correct: wf-fmla-atom1' STG φ = wf-fmla-atom ty-ent φ
 <proof>

```

fun wf-effect' where
  wf-effect' stg (Effect a d)  $\longleftrightarrow$ 
    ( $\forall ae \in \text{set } a. \text{ wf-fmla-atom1' stg } ae$ )
     $\wedge$  ( $\forall de \in \text{set } d. \text{ wf-fmla-atom1' stg } de$ )

```

lemma wf-effect'-correct: wf-effect' STG e = wf-effect ty-ent e
 <proof>

```

fun wf-duration-const' :: -  $\Rightarrow$  'ent duration-constraint  $\Rightarrow$  bool where
  wf-duration-const' stg No-Const  $\longleftrightarrow$  True
| wf-duration-const' stg (Time-Const op d)  $\longleftrightarrow$   $d \geq 0$ 
| wf-duration-const' stg (Func-Const op f vs)  $\longleftrightarrow$ 
  (case func-sig f of

```

$None \Rightarrow False$
 $| Some\ Ts \Rightarrow list-all2\ (is-of-type'\ stg)\ vs\ Ts$

lemma *wf-duration-const'-correct*: *wf-duration-const' STG d = wf-duration-const*
ty-ent d
 $\langle proof \rangle$

definition *wf-duration-consts' stg = list-all (wf-duration-const' stg)*

lemma *wf-duration-consts'-correct*: *wf-duration-consts' STG ds = wf-duration-consts*
ty-ent ds
 $\langle proof \rangle$
end — Context fixing *ty-ent*

fun *wf-action-schema' :: - $\Rightarrow$ - $\Rightarrow$ ast-action-schema $\Rightarrow$ bool* **where**
wf-action-schema' stg conT (Simple-Action-Schema n params pre eff) $\longleftrightarrow$ (
let
tyv = ty-term' (map-of params) conT
in
distinct (map fst params)
 $\wedge$ *wf-fmla' tyv stg pre*
 $\wedge$ *wf-effect' tyv stg eff)*
 $| wf-action-schema' stg conT (Durative-Action-Schema n params d cond eff) \longleftrightarrow$
 $($
let
tyv = ty-term' (map-of params) conT
in
distinct (map fst params)
 $\wedge (\forall (t,c) \in set\ cond.\ wf-fmla'\ tyv\ stg\ c)$
 $\wedge (\forall (t,e) \in set\ eff.\ wf-effect'\ tyv\ stg\ e \wedge t \neq Over-All)$
 $\wedge wf-duration-consts'\ tyv\ stg\ d)$

lemma *wf-action-schema'-correct*: *wf-action-schema' STG mp-constT s = wf-action-schema*
s
 $\langle proof \rangle$

definition *wf-domain' :: - $\Rightarrow$ - $\Rightarrow$ bool* **where**
wf-domain' stg conT $\equiv$
wf-types
 $\wedge distinct\ (map\ (predicate-decl.pred)\ (predicates\ D))$
 $\wedge (\forall p \in set\ (predicates\ D).\ wf-predicate-decl\ p)$
 $\wedge distinct\ (map\ (function-decl.func)\ (functions\ D))$
 $\wedge (\forall f \in set\ (functions\ D).\ wf-function-decl\ f)$
 $\wedge distinct\ (map\ fst\ (consts\ D))$
 $\wedge (\forall (n,T) \in set\ (consts\ D).\ wf-type\ T)$
 $\wedge distinct\ (map\ ast-action-schema.name\ (actions\ D))$
 $\wedge (\forall a \in set\ (actions\ D).\ wf-action-schema'\ stg\ conT\ a)$

lemma *wf-domain'-correct*: *wf-domain' STG mp-constT = wf-domain*
<proof>

end — Context of *ast-domain*

2.2.2 Ground Action Interference & Application of Effects

context *ast-problem* **begin**

Implementation of executable refinements for *acts-non-intrf* & *apply-happ*

a list of simulataneous & non-interfering grounded actions are all applied at once

fun *apply-happ-exec* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ *world-model* **where**
apply-happ-exec (*t_i*, *A_i*) *s* =
 (let
adds = (*concat o (map (adds o effect))*) *A_i*;
dels = (*concat o (map (dels o effect))*) *A_i* in
fold Set.insert adds (fold Set.remove dels s))

lemma *fold-set-remove*: *fold Set.remove xs s = s - set xs*
<proof>

lemma *fold-set-insert*: *fold Set.insert xs s = s $\cup$ set xs*
<proof>

Justification of refinement for *apply-happ*

lemma *apply-happ-exec-refine*: *apply-happ-exec h s = apply-happ h s*
<proof>

end — Context of *ast-problem*

2.2.3 Well-Formedness

context *ast-problem* **begin**

We start by defining a mapping from objects to types. The container framework will generate efficient, red-black tree based code for that later.

type-synonym *objT* = (*object*, *type*) *mapping*

definition *mp-objT* :: (*object*, *type*) *mapping* **where**
mp-objT = *Mapping.of-alist (consts D @ objects P)*

lemma *mp-objT-correct[simp]*: *Mapping.lookup mp-objT = objT*
<proof>

We refine the typecheck to use the mapping

definition *is-obj-of-type-impl stg mp n T* = (

case Mapping.lookup mp n of None $\Rightarrow$ False | Some oT $\Rightarrow$ of-type-impl stg oT
T
)

lemma is-obj-of-type-impl-correct[simp]:
is-obj-of-type-impl STG mp-objT = is-obj-of-type
⟨proof⟩

fun wf-func-assign' :: objT $\Rightarrow$ - $\Rightarrow$ object atom formula $\Rightarrow$ bool **where**
wf-func-assign' ot stg (Atom (eqAtm (FuncEnt f args) (TimeEnt d)))
 $\longleftrightarrow$ wf-func-args' (Mapping.lookup ot) stg (f,args)
| wf-func-assign' ot stg - $\longleftrightarrow$ False

lemma wf-func-assign'-correct[simp]: wf-func-assign' mp-objT STG f = wf-func-assign
f
⟨proof⟩

We refine the well-formedness checks to use the mapping

definition wf-fact' :: objT $\Rightarrow$ - $\Rightarrow$ fact $\Rightarrow$ bool **where**
wf-fact' ot stg $\equiv$ wf-pred-atom' (Mapping.lookup ot) stg

lemma wf-fact'-correct[simp]: wf-fact' mp-objT STG = wf-fact
⟨proof⟩

definition wf-fmla-atom2' mp stg f
= (case f of formula.Atom (predAtm p vs) $\Rightarrow$ (wf-fact' mp stg (p,vs)) | - $\Rightarrow$
False)

lemma wf-fmla-atom2'-correct[simp]:
wf-fmla-atom2' mp-objT STG φ = wf-fmla-atom objT φ
⟨proof⟩

definition wf-problem' stg conT mp $\equiv$
wf-domain' stg conT
 $\wedge$ distinct (map fst (objects P) @ map fst (consts D))
 $\wedge$ ($\forall (n,T) \in \text{set } (\text{objects } P). \text{wf-type } T$)
 $\wedge$ distinct (init P)
 $\wedge$ ($\forall f \in \text{set } (\text{init } P). \text{wf-fmla-atom2' mp stg } f \vee \text{wf-func-assign' mp stg } f$)
 $\wedge$ wf-fmla' (Mapping.lookup mp) stg (goal P)

lemma wf-problem'-correct:
wf-problem' STG mp-constT mp-objT = wf-problem
⟨proof⟩

Instantiating actions will yield well-founded effects. Corollary of $\llbracket \text{action-params-match} \text{ (Simple-Action-Schema ?n ?params ?pre ?eff) ?args; wf-action-schema (Simple-Action-Schema ?n ?params ?pre ?eff)} \rrbracket \implies \text{wf-ground-action (instantiate-action-schema (Simple-Action-Schema ?n ?params ?pre ?eff) ?args ?ta)}$ and $\llbracket \text{action-params-match}$

$(\text{Durative-Action-Schema } ?n \text{ } ?params \text{ } ?d \text{ } ?cond \text{ } ?eff) \text{ } ?args; \text{wf-action-schema } (\text{Durative-Action-Schema } ?n \text{ } ?params \text{ } ?d \text{ } ?cond \text{ } ?eff) \parallel \implies \text{wf-ground-action } (\text{inst-snap-action } (\text{Durative-Action-Schema } ?n \text{ } ?params \text{ } ?d \text{ } ?cond \text{ } ?eff) \text{ } ?args \text{ } ?ta).$

lemma *wf-effect-inst-weak*:
fixes $n \text{ } params \text{ } pre \text{ } eff$
defines $a_{schema} \equiv \text{Simple-Action-Schema } n \text{ } params \text{ } pre \text{ } eff$
assumes $a = \text{instantiate-action-schema } a_{schema} \text{ } args \text{ } ta$
and $\text{action-params-match } a_{schema} \text{ } args$
and $\text{wf-action-schema } a_{schema}$
shows $\text{wf-effect-inst } (effect \text{ } a)$
 $\langle proof \rangle$

lemma *wf-effect-durative-inst-weak*:
fixes $n \text{ } params \text{ } d \text{ } cond \text{ } eff$
defines $a_{schema} \equiv \text{Durative-Action-Schema } n \text{ } params \text{ } d \text{ } cond \text{ } eff$
assumes $a = \text{inst-snap-action } a_{schema} \text{ } args \text{ } ta$
and $\text{action-params-match } a_{schema} \text{ } args$
and $\text{wf-action-schema } a_{schema}$
shows $\text{wf-effect-inst } (effect \text{ } a)$
 $\langle proof \rangle$

end — Context of *ast-problem*

2.2.4 Happening Execution

context *ast-domain* **begin**

We first lift action schema lookup into the error monad.

definition *resolve-action-schemaE* $n \equiv$
 lift-opt
 $(\text{resolve-action-schema } n)$
 $(\text{ERR } (\text{shows "No such action schema " } o \text{ shows } n))$

lemma *resolve-action-schemaE-return-iff*[*return-iff*]:
 $(\text{resolve-action-schemaE } n = \text{Inr } a) \longleftrightarrow (\text{resolve-action-schema } n = \text{Some } a)$
 $\langle proof \rangle$

end — Context of *ast-domain*

context *ast-problem* **begin**

We define a function to determine whether a formula holds in a world model

definition *holds* $M \text{ } F \equiv (\text{valuation } M) \models F$

Justification of this function

lemma *holds-for-wf-fmlas*:
assumes $wm\text{-basic } s$

shows *holds* $s \ F \longleftrightarrow \text{close-world } s \models F$
 $\langle \text{proof} \rangle$

The first refinement summarizes the enabledness check and the application of a happening. Moreover, we implement the precondition evaluation by our *holds* function. This way, we can eliminate redundant resolving and instantiation of the action.

definition *en-exE* :: *happening* $\Rightarrow$ *world-model* $\Rightarrow$ $- +$ *world-model* **where**
en-exE $\equiv \lambda(t_i, A_i) \Rightarrow \lambda s. \text{do} \{$
 check-allm ($\lambda a. \text{check} (\text{holds } s \text{ (precondition } a))$) (*ERRS* "Precondition not satisfied") A_i ;
 check-pairwise ($\lambda a b. \text{check} (a \neq b \longrightarrow \text{acts-non-intrf } a \ b)$) (*ERRS* "Actions in happening interfering") A_i ;
 Error-Monad.return (*apply-happ-exec* (t_i, A_i) s)
 $\}$

lemma *symmetric-pred-check-pairwise*:

assumes $\forall x_1 \ x_2. A \ x_1 \ x_2 \longleftrightarrow A \ x_2 \ x_1$
and *isOK* (*check-pairwise* ($\lambda x_1 \ x_2. \text{check} (x_1 \neq x_2 \longrightarrow A \ x_1 \ x_2)$) e) *as*)
shows $\forall x_1 \in \text{set } as. \forall x_2 \in \text{set } as. x_1 \neq x_2 \longrightarrow A \ x_1 \ x_2$
 $\langle \text{proof} \rangle$

Justification of implementation.

lemma (**in** *wf-ast-problem*) *en-exE-return-iff*:

assumes *wm-basic* s
and $\forall a \in \text{set } A_i. \text{wf-ground-action } a$
shows *en-exE* (t_i, A_i) $s = \text{Inr } s' \longleftrightarrow \text{happ-enabled } (t_i, A_i) \ s \wedge s' = \text{apply-happ} (t_i, A_i) \ s$
 $\langle \text{proof} \rangle$

Next, we use the efficient implementation *is-obj-of-type-impl* for the type check, and omit the well-formedness check, as effects obtained from instantiating well-formed action schemas are always well-formed (*wf-effect-inst-weak*).

abbreviation *action-params-match2* *stg mp a args*
 $\equiv \text{list-all2} (\text{is-obj-of-type-impl } \text{stg } \text{mp})$
 $\text{args } (\text{map } \text{snd } (\text{ast-action-schema.parameters } a))$

end — Context of *ast-problem*

2.2.5 Construction of Induced Happening Sequence

context *ast-problem* **begin**

implemented own version of *insort-insert*, because *insort-insert* requires sort type; rat is not of sort type

fun *insort-htp* :: *time* $\Rightarrow$ *time list* $\Rightarrow$ *time list* **where**
 insort-htp $x \ [] = [x]$
 | *insort-htp* $x \ (y \# \text{ys}) =$

(if $x < y$ then $x \# y \# ys$
 else if $x = y$ then $y \# ys$
 else $y \# (\text{insort-htp } x \text{ } ys)$)

Basic properties text of *insort-htp*: preserves sorted-ness & distinct-ness, inserts element

lemma *insort-htps-insort-insert*: $\text{sorted } xs \implies \text{insort-htp } x \text{ } xs = \text{insort-insert } x \text{ } xs$
 $\langle \text{proof} \rangle$

lemma *sorted-insort-htps*: $\text{sorted } xs \implies \text{sorted } (\text{insort-htp } x \text{ } xs)$
 $\langle \text{proof} \rangle$

lemma *set-insort-htps*: $\text{set } (\text{insort-htp } x \text{ } xs) = \text{insert } x \text{ } (\text{set } xs)$
 $\langle \text{proof} \rangle$

lemma *distinct-insort-htps*: $\text{sorted } xs \implies \text{distinct } xs \implies \text{distinct } (\text{insort-htp } x \text{ } xs)$
 $\langle \text{proof} \rangle$

lemma *insort-htps-non-Nil*: $\text{insort-htp } x \text{ } xs \neq []$
 $\langle \text{proof} \rangle$

Implementation of executable refinement for happening time points. Produces a strictly sorted sequence of all *is-htp*

fun *htps-exec* :: *plan* $\Rightarrow$ *time list* **where**
htps-exec [] = []
 | *htps-exec* ((t_π , *Simple-Plan-Action* n *args*) # πs) =
 insort-htp t_π (*htps-exec* πs)
 | *htps-exec* ((t_π , *Durative-Plan-Action* n *args* d) # πs) =
 insort-htp t_π (*insort-htp* ($t_\pi + d$) (*htps-exec* πs))

Basic properties of *htps-exec*

lemma *htps-exec-Nil*: $\text{htps-exec } \pi s = [] \longleftrightarrow \pi s = []$
 $\langle \text{proof} \rangle$

lemma *htps-exec-sorted*: $\text{sorted } (\text{htps-exec } \pi s)$
 $\langle \text{proof} \rangle$

lemma *htps-exec-distinct*: $\text{distinct } (\text{htps-exec } \pi s)$
 $\langle \text{proof} \rangle$

Correctness proof for *htps-exec*

Every time point in *htps-exec* is a happening time point.

lemma *htps-exec-is-htps*: $t_i \in \text{set } (\text{htps-exec } \pi s) \implies \text{is-htp } \pi s \text{ } t_i$
 $\langle \text{proof} \rangle$

lemma *https-exec-Cons-subset*: $set\ (https-exec\ \pi s) \subseteq set\ (https-exec\ (\pi \# \pi s))$
 $\langle proof \rangle$

lemma *https-exec-app-subset*: $set\ (https-exec\ \pi s_2) \subseteq set\ (https-exec\ (\pi s_1 @ \pi s_2))$
 $\langle proof \rangle$

Every happening time point is included in *https-exec*.

lemma *is-http-https-exec*: $is-http\ \pi s\ t_i \implies t_i \in set\ (https-exec\ \pi s)$
 $\langle proof \rangle$

lemma *is-http-iff-https-exec*: $\forall t. is-http\ \pi s\ t \longleftrightarrow t \in set\ (https-exec\ \pi s)$
 $\langle proof \rangle$

definition *consec-https-in-interval* :: $time\ list \Rightarrow time \Rightarrow time \Rightarrow (time \times time)$
list where
 $consec-https-in-interval\ https\ t_i\ t_j =$
 $filter\ (\lambda(t, t'). t_i \leq t \wedge t' \leq t_j)\ (zip\ (butlast\ https)\ (tl\ https))$

Construct all ground actions for a given plan action

fun *simplify-action* :: $time\ list \Rightarrow (time \times plan-action) \Rightarrow (time \times ground-action)$
list where
 $simplify-action\ https\ (t, Simple-Plan-Action\ n\ args) = ($
 $let\ a = the\ (resolve-action-schema\ n)\ in$
 $[(t, instantiate-action-schema\ a\ args\ At-Start)]$
 $)$
 $| simplify-action\ https\ (t, Durative-Plan-Action\ n\ args\ d) = ($
 $let\ a = the\ (resolve-action-schema\ n)\ in$
 $(t, inst-snap-action\ a\ args\ At-Start) \#$
 $(t+d, inst-snap-action\ a\ args\ At-End) \#$
 $(map\ (\lambda(t_i, t_j). ((t_i + t_j) / 2, inst-snap-action\ a\ args\ Over-All))\ (consec-https-in-interval$
 $https\ t\ (t+d)))$
 $)$

Auxiliary lemmas to prove properties about ' $zip\ (butlast\ xs)\ (tl\ xs)$ ', which are needed to ultimately prove properties about *consec-https*.

lemma *zip-butlast-tl-hd*:
assumes *distinct* $(x_1 \# xs)$
and $(x_1, x_2) \in set\ (zip\ (butlast\ (x_1 \# xs))\ (tl\ (x_1 \# xs)))$
shows $xs \neq [] \wedge x_2 = hd\ xs$
 $\langle proof \rangle$

lemma *zip-butlast-sublist*:
assumes *distinct* $(xs_1 @ x_1 \# xs_2)$
and $(x_1, x_2) \in set\ (zip\ (butlast\ (xs_1 @ x_1 \# xs_2))\ (tl\ (xs_1 @ x_1 \# xs_2)))$
shows $(x_1, x_2) \in set\ (zip\ (butlast\ (x_1 \# xs_2))\ (tl\ (x_1 \# xs_2)))$
 $\langle proof \rangle$

lemma *zip-butlast-tl-app1*:
assumes *distinct* xs

and $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) (\text{tl } xs))$
shows $\exists xs_1 \ xs_2. xs = xs_1 @ x_1 \# x_2 \# xs_2$
 $\langle \text{proof} \rangle$

lemma *zip-butlast-tl-app2*:
 $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } (xs_1 @ x_1 \# x_2 \# xs_2)) (\text{tl } (xs_1 @ x_1 \# x_2 \# xs_2)))$
 $\langle \text{proof} \rangle$

lemma *zip-butlast-tl-app*:
assumes *distinct xs*
shows $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) (\text{tl } xs)) \longleftrightarrow (\exists xs_1 \ xs_2. xs = xs_1 @ x_1 \# x_2 \# xs_2)$
 $\langle \text{proof} \rangle$

lemma *sorted-distinct-zip-le*:
assumes *strict-sorted xs*
and $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) (\text{tl } xs))$
shows $x_1 < x_2$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-zip-butlast-tl-leq-or-geq*:
assumes *strict-sorted xs*
and $(x_1, x_2) \in \text{set } (\text{zip } (\text{butlast } xs) (\text{tl } xs))$
and $x \in \text{set } xs$
shows $x \leq x_1 \vee x_2 \leq x$
 $\langle \text{proof} \rangle$

lemma *consec-https-in-zip-butlast-tl*:
assumes $https = https\text{-exec } \pi s$
and $\text{consec-https } \pi s \ t_i \ t_j$
shows $(t_i, t_j) \in \text{set } (\text{zip } (\text{butlast } https) (\text{tl } https))$
 $\langle \text{proof} \rangle$

Executable refinement for *consec-https*.

lemma *happ-time-pts-exec-iff-consec-https*:
 $(t_i, t_j) \in \text{set } (\text{zip } (\text{butlast } (https\text{-exec } \pi s)) (\text{tl } (https\text{-exec } \pi s))) \longleftrightarrow \text{consec-https } \pi s \ t_i \ t_j$
 $\langle \text{proof} \rangle$

Justification for *consec-https-in-interval*.

lemma *consec-https-in-interval-iff*:
assumes $t_i \leq t_j$
shows $\forall t \ t'. (\text{consec-https } \pi s \ t \ t' \wedge t_i \leq t \wedge t' \leq t_j) \longleftrightarrow (t, t') \in \text{set } (\text{consec-https-in-interval } (https\text{-exec } \pi s) \ t_i \ t_j)$
 $\langle \text{proof} \rangle$

All returned actions from *simplify-action* are correct instantiations of a *plan-action* for an induced simple plan.

lemma (in *wf-ast-problem*) *simp-act-inst-of-plan-act*:

assumes *wf-plan* πs
and $(t_\pi, \pi) \in \text{set } \pi s$
and $(t_a, a) \in \text{set } (\text{simplify-action } (\text{https-exec } \pi s) (t_\pi, \pi))$
shows *inst-of-plan-action* $\pi s (t_\pi, \pi) (t_a, a)$
 $\langle \text{proof} \rangle$

Function to insert a happening into a happening sequence.

fun *insort-happ* :: *happening* $\Rightarrow$ *happening list* $\Rightarrow$ *happening list* **where**
insort-happ $(t_i, A_i) [] = [(t_i, A_i)]$
| *insort-happ* $(t_i, A_i) ((t_j, A_j) \# hs) =$
 (if $t_i < t_j$ then $(t_i, A_i) \# (t_j, A_j) \# hs$
 else if $t_i = t_j$ then $(t_j, A_i @ A_j) \# hs$
 else $(t_j, A_j) \# (\text{insort-happ } (t_i, A_i) hs)$)

Properties of *insort-happ*

lemma *insort-happ-insort-insert*:
 $\text{sorted } (\text{map fst } hs) \implies \text{insort-insert } (\text{fst } h) (\text{map fst } hs) = \text{map fst } (\text{insort-happ } h hs)$
 $\langle \text{proof} \rangle$

insort-happ preserves sorted-ness.

lemma *insort-happ-distinct*: $\text{strict-sorted } (\text{map fst } hs) \implies \text{distinct } (\text{map fst } (\text{insort-happ } h hs))$
 $\langle \text{proof} \rangle$

lemma *insort-happ-not-Nil*:
assumes $A_i \neq []$ **and** $\forall (t_j, A_j) \in \text{set } hs. A_j \neq []$
shows $\forall (t_j, A_j) \in \text{set } (\text{insort-happ } (t_i, A_i) hs). A_j \neq []$
 $\langle \text{proof} \rangle$

lemma *insort-happ-action-subset-relaxed*:
assumes $(\exists H''. (t, H'') \in \text{set } hs \wedge \text{set } H \subseteq \text{set } H'') \vee \text{set } H \subseteq \text{set } H'$
shows $\exists A. (t, A) \in \text{set } (\text{insort-happ } (t, H') hs) \wedge \text{set } H \subseteq \text{set } A$
 $\langle \text{proof} \rangle$

lemma *insort-happ-action-subset*:
 $\forall (t_a, A) \in \text{set } hs \cup \{h\}. \exists A'. (t_a, A') \in \text{set } (\text{insort-happ } h hs) \wedge \text{set } A \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *insort-happ2-action-subset*:
 $\exists A'. (t_2, A') \in \text{set } (\text{insort-happ } (t_1, A_1) (\text{insort-happ } (t_2, A_2) hs)) \wedge \text{set } A_2 \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *insort-happ-sorted*: $\text{sorted } (\text{map } \text{fst } \text{hs}) \implies \text{sorted } (\text{map } \text{fst } (\text{insort-happ } h \text{ hs}))$

$\langle \text{proof} \rangle$

lemma *insort-happ-strict-sorted*:

$\text{strict-sorted } (\text{map } \text{fst } \text{hs}) \implies \text{strict-sorted } (\text{map } \text{fst } (\text{insort-happ } h \text{ hs}))$

$\langle \text{proof} \rangle$

Function to insert multiple happening into a happening sequence.

fun *insort-mult-happs* :: $\text{happening list} \Rightarrow \text{happening list} \Rightarrow \text{happening list}$ **where**

insort-mult-happs [] $\text{hs}_2 = \text{hs}_2$

| *insort-mult-happs* ($h \# \text{hs}_1$) $\text{hs}_2 = \text{insort-happ } h (\text{insort-mult-happs } \text{hs}_1 \text{ hs}_2)$

fun *insort-timed-ground-acts* :: $(\text{time} \times \text{ground-action}) \text{ list} \Rightarrow \text{happening list} \Rightarrow \text{happening list}$ **where**

insort-timed-ground-acts [] $\text{hs} = \text{hs}$

| *insort-timed-ground-acts* ($(t_a, a) \# \text{as}$) $\text{hs} =$

insort-happ ($t_a, [a]$) (*insort-timed-ground-acts* as hs)

lemma *insort-timed-ground-acts-correct*:

insort-mult-happs ($\text{map } (\lambda(t, a). (t, [a])) \text{ as}$) $\text{hs} = \text{insort-timed-ground-acts as hs}$

$\langle \text{proof} \rangle$

Properties of *insort-mult-happs*

lemma *insort-mult-happs-strict-sorted*:

$\text{strict-sorted } (\text{map } \text{fst } \text{hs}_2) \implies \text{strict-sorted } (\text{map } \text{fst } (\text{insort-mult-happs } \text{hs}_1 \text{ hs}_2))$

$\langle \text{proof} \rangle$

lemma *insort-mult-happs-not-Nil*:

assumes $\forall (t_a, A) \in \text{set } \text{xs}. A \neq []$ **and** $\forall (t_a, A) \in \text{set } \text{ys}. A \neq []$

shows $\forall (t_a, A) \in \text{set } (\text{insort-mult-happs } \text{xs } \text{ys}). A \neq []$

$\langle \text{proof} \rangle$

lemma *insort-mult-happs-action-subset1*:

$(t_a, A) \in \text{set } \text{hs}_1 \implies \exists \text{as}'. (t_a, \text{as}') \in \text{set } (\text{insort-mult-happs } \text{hs}_1 \text{ hs}_2) \wedge \text{set } A \subseteq \text{set } \text{as}'$

$\langle \text{proof} \rangle$

lemma *insort-mult-happs-action-subset2*:

assumes $(t_a, A) \in \text{set } \text{hs}_2$

shows $\exists A'. (t_a, A') \in \text{set } (\text{insort-mult-happs } \text{hs}_1 \text{ hs}_2) \wedge \text{set } A \subseteq \text{set } A'$

$\langle \text{proof} \rangle$

lemma *insort-mult-happs-action-subset*:

$\forall (t_a, A) \in \text{set } \text{hs}_1 \cup \text{set } \text{hs}_2. \exists A'. (t_a, A') \in \text{set } (\text{insort-mult-happs } \text{hs}_1 \text{ hs}_2) \wedge \text{set } A \subseteq \text{set } A'$

$\langle \text{proof} \rangle$

lemma *insort-happ2-insort-mult-happs-action-subset-aux*:
assumes $(t_a, A) \in \text{set } hs_1$
shows $\exists A'. (t_a, A') \in \text{set } (\text{insort-happ } (t_1, A_1) (\text{insort-happ } (t_2, A_2) (\text{insort-mult-happs } hs_1 \ hs_2))) \wedge \text{set } A \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

lemma *insort-happ2-insort-mult-happs-action-subset*:
 $\forall (t_a, A) \in \text{set } hs_1. \exists A'. (t_a, A') \in \text{set } (\text{insort-happ } (t_1, A_1) (\text{insort-happ } (t_2, A_2) (\text{insort-mult-happs } hs_1 \ hs_2))) \wedge \text{set } A \subseteq \text{set } A'$
 $\langle \text{proof} \rangle$

We can define a function to construct a simplified plan for a given temporal plan.

fun *simplify-plan* :: *time list* $\Rightarrow$ *plan* $\Rightarrow$ *happening list* **where**
simplify-plan *htps* [] = []
| *simplify-plan* *htps* ($\pi \# \pi s$) =
insort-mult-happs ($\text{map } (\lambda(t_a, a). (t_a, [a])) (\text{simplify-action } \text{htps } \pi)$) (*simplify-plan* *htps* πs)

Definition of subset on happening sequences. This is used to simplify some proofs.

definition *happ-seq-member* :: *happening* $\Rightarrow$ *happening list* $\Rightarrow$ *bool* (**infix** $\in_{hs}$ 53) **where**
happ-seq-member *h* *hs* $\longleftrightarrow (\text{case } h \text{ of } (t_a, A) \Rightarrow \exists A_2. \text{set } A \subseteq \text{set } A_2 \wedge (t_a, A_2) \in \text{set } hs)$

definition *happ-seq-subset* :: *happening list* $\Rightarrow$ *happening list* $\Rightarrow$ *bool* (**infix** $\subseteq_{hs}$ 53) **where**
happ-seq-subset *hs*₁ *hs*₂ $\longleftrightarrow (\forall h_1 \in \text{set } hs_1. h_1 \in_{hs} hs_2)$

Properties of ($\subseteq_{hs}$)

lemma *happ-seq-subset-Nil*: [] $\subseteq_{hs}$ *hs*
 $\langle \text{proof} \rangle$

lemma *happ-seq-subset-refl*: *hs* $\subseteq_{hs}$ *hs*
 $\langle \text{proof} \rangle$

lemma *happ-seq-subset-trans*: *hs*₁ $\subseteq_{hs}$ *hs*₂ $\wedge$ *hs*₂ $\subseteq_{hs}$ *hs*₃ $\implies$ *hs*₁ $\subseteq_{hs}$ *hs*₃
 $\langle \text{proof} \rangle$

lemma *insort-happ-subset*: *hs* $\subseteq_{hs}$ *insort-happ* *h* *hs*
 $\langle \text{proof} \rangle$

lemma *insort-mult-happs-subset*: *hs*₁ $\subseteq_{hs}$ *insort-mult-happs* *hs*₂ *hs*₁
 $\langle \text{proof} \rangle$

Happening sequence subset for simplify plan.

lemma *simplify-plan-happs-subset-Cons*: *simplify-plan https $\pi s \subseteq_{hs}$ simplify-plan https ($\pi \# \pi s$)*
<proof>

lemma *simplify-plan-happs-subset-app*: *simplify-plan https $\pi s_2 \subseteq_{hs}$ simplify-plan https ($\pi s_1 @ \pi s_2$)*
<proof>

Properties of *simplify-plan*

All happenings in a simplified plan are always non-*Nil*.

lemma *simp-plan-happ-not-Nil*:
fixes πs
defines $https \equiv https-exec \pi s$
shows $\forall (t_a, A) \in set (simplify-plan https \pi s). A \neq []$
<proof>

A simplified plan produced by *simplify-plan* is strictly sorted.

lemma *simp-plan-strict-sorted*: *strict-sorted (map fst (simplify-plan https πs))*
<proof>

A simplified plan contains all grounded actions for every *Simple-Plan-Action*.

lemma *simp-plan-corr-simp-acts*:
assumes $hs = simplify-plan https \pi s$
and $(t_\pi, \pi) \in simple-acts \pi s$
shows $\exists A. (t_\pi, A) \in set hs \wedge the (res-inst \pi At-Start) \in set A$
<proof>

lemma *simp-plan-corr-dur-acts-start-aux*:
assumes $hs = simplify-plan https \pi s$
and $(t_\pi, \pi) = hd \pi s$
and $(t_\pi, \pi) \in durative-acts \pi s$
shows $(\exists A. (t_\pi, A) \in set hs \wedge (the (res-inst-snap-action \pi At-Start)) \in set A)$
<proof>

A simplified plan contains all *At-Start-snap-actions*.

lemma *simp-plan-corr-dur-acts-start*:
fixes π
defines $a_{start} \equiv the (res-inst-snap-action \pi At-Start)$
assumes $hs = simplify-plan https \pi s$
and $(t_\pi, \pi) \in durative-acts \pi s$
shows $\exists A. (t_\pi, A) \in set hs \wedge a_{start} \in set A$
<proof>

lemma *simp-plan-corr-dur-acts-end-aux*:
assumes $hs = simplify-plan https \pi s$
and $(t_\pi, \pi) = hd \pi s$
and $(t_\pi, \pi) \in durative-acts \pi s$

shows $(\exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge (\text{the } (\text{res-inst-snap-action } \pi \text{ At-End})) \in \text{set } A)$
 $\langle \text{proof} \rangle$

A simplified plan contains all *At-End*-snap-actions.

lemma *simp-plan-corr-dur-acts-end*:
fixes π
defines $a_{end} \equiv \text{the } (\text{res-inst-snap-action } \pi \text{ At-End})$
assumes $hs = \text{simplify-plan } htps \ \pi s$
and $(t_\pi, \pi) \in \text{durative-acts } \pi s$
shows $\exists A. (t_\pi + (\text{duration } \pi), A) \in \text{set } hs \wedge a_{end} \in \text{set } A$
 $\langle \text{proof} \rangle$

lemma *member-map*: $a \in \text{set } A \implies f a \in \text{set } (\text{map } f A)$
 $\langle \text{proof} \rangle$

lemma *simp-plan-corr-dur-acts-overall-aux*:
assumes *strict-sorted* $htps$
and $hs = \text{simplify-plan } htps \ \pi s$
and $(t_\pi, \pi) = \text{hd } \pi s$
and $(t_\pi, \pi) \in \text{durative-acts } \pi s$
shows $\forall t_i t_j. (t_i, t_j) \in \text{set } (\text{consec-htps-in-interval } htps \ t_\pi \ (t_\pi + \text{duration } \pi))$
 $\implies$
 $(\exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge (\text{the } (\text{res-inst-snap-action } \pi \text{ Over-All})) \in \text{set } A)$
 $\langle \text{proof} \rangle$

A simplified plan contains all *Over-All*-snap-actions.

lemma (*in wf-ast-problem*) *simp-plan-corr-dur-acts-overall*:
fixes $\pi \ \pi s$
defines $htps \equiv \text{htps-exec } \pi s$
and $a_{inv} \equiv \text{the } (\text{res-inst-snap-action } \pi \text{ Over-All})$
assumes *wf-plan* πs
and $hs = \text{simplify-plan } htps \ \pi s$
and $(t_\pi, \pi) \in \text{durative-acts } \pi s$
shows $\forall t_i t_j. (\text{consec-htps } \pi s \ t_i \ t_j \wedge t_\pi \leq t_i \wedge t_j \leq t_\pi + \text{duration } \pi) \implies$
 $(\exists (t', A) \in \text{set } hs. t_i < t' \wedge t' < t_j \wedge a_{inv} \in \text{set } A)$
 $\langle \text{proof} \rangle$

Flatten a happening into individual timed grounded actions. This is used to simplify proofs.

fun *set-happ-acts* :: *happening list* $\Rightarrow$ (*time* $\times$ *ground-action*) *set* **where**
 $\text{set-happ-acts } [] = \{\}$
 $| \text{set-happ-acts } ((t_a, A) \# hs) = \{(t_a', a'). t_a = t_a' \wedge a' \in \text{set } A\} \cup \text{set-happ-acts } hs$

lemma *set-happ-acts-refine*:
 $(\exists A. (t_a, A) \in \text{set } hs \wedge a \in \text{set } A) \longleftrightarrow ((t_a, a) \in \text{set-happ-acts } hs)$

$\langle proof \rangle$

lemma *set-happ-acts-insort-happ*:

set-happ-acts (insort-happ h hs) = set-happ-acts [h] $\cup$ set-happ-acts hs

$\langle proof \rangle$

lemma *set-happ-acts-insort-mult-happs*:

set-happ-acts (insort-mult-happs hs₁ hs₂) = set-happ-acts hs₁ $\cup$ set-happ-acts hs₂

$\langle proof \rangle$

lemma *set-happ-acts-to-happ*: *set-happ-acts (map ($\lambda(t_a, a). (t_a, [a])$) xs) = set xs*

$\langle proof \rangle$

lemma *simpl-plan-corr-inst-plan-act-aux*:

assumes *strict-sorted htps*

and *hs = simplify-plan htps π s*

shows $\forall (t_a, a) \in \text{set-happ-acts } hs. \exists (t_\pi, \pi) \in \text{set } \pi s. (t_a, a) \in \text{set (simplify-action htps (t_\pi, \pi))}$

$\langle proof \rangle$

Every ground action in a simplified plan is the instantiation of a plan action.

lemma (in *wf-ast-problem*) *simpl-plan-corr-inst-plan-act*:

fixes πs

defines *htps* $\equiv$ *htps-exec π s*

assumes *wf-plan π s*

and *hs = simplify-plan htps π s*

and $(t_a, A) \in \text{set } hs$

and $a \in \text{set } A$

shows $\exists (t_\pi, \pi) \in \text{set } \pi s. \text{inst-of-plan-action } \pi s (t_\pi, \pi) (t_a, a)$

$\langle proof \rangle$

Finally the correctness proof for *simplify-plan*

lemma (in *wf-ast-problem*) *simplify-plan-correct*:

fixes πs

defines *htps* $\equiv$ *htps-exec π s*

assumes *wf-plan π s*

shows *ind-happ-seq π s (simplify-plan htps π s)*

$\langle proof \rangle$

lemma (in *wf-ast-problem*)

assumes *wf-plan π s*

shows *ind-happ-seq π s (simplify-plan (htps-exec π s) π s)*

$\langle proof \rangle$

With the correctness proof of *simplify-plan*. We can easily proof that for every plan there exists an induced happening sequence.

lemma (in *wf-ast-problem*) *ind-happ-seq-exists*:

assumes *wf-plan* πs
shows $\exists hs. \text{ind-happ-seq } \pi s \ hs$
 $\langle \text{proof} \rangle$

lemma (in *wf-ast-problem*) *simplify-plan-wf-ground-action-aux*:
fixes πs
defines $https \equiv https\text{-exec } \pi s$
assumes *wf-plan* πs
and $hs = \text{simplify-plan } https \ \pi s$
and $(t_a, A) \in \text{set } hs$
and $a \in \text{set } A$
shows *wf-ground-action* a
 $\langle \text{proof} \rangle$

lemma (in *wf-ast-problem*) *simplify-plan-wf-ground-action*:
fixes πs
defines $https \equiv https\text{-exec } \pi s$
assumes *wf-plan* πs
and $hs = \text{simplify-plan } https \ \pi s$
shows $\forall (t_a, A) \in \text{set } hs. \forall a \in \text{set } A. \text{wf-ground-action } a$
 $\langle \text{proof} \rangle$

Recursive refinement for *valid-happ-seq*.

fun *valid-happ-seq-from* :: *world-model* $\Rightarrow$ *happening list* $\Rightarrow$ *bool* **where**
valid-happ-seq-from $s \ [] \longleftrightarrow \text{close-world } s \models (\text{goal } P)$
 $| \text{valid-happ-seq-from } s \ (h \# hs) \longleftrightarrow$
 $(\text{happ-enabled } h \ s \wedge \text{valid-happ-seq-from } (\text{apply-happ-exec } h \ s) \ hs)$

Justification of the refinement for *valid-happ-seq*.

lemma *valid-happ-seq-from-refine*:
assumes *wf-happ-seq* hs
shows $\text{valid-happ-seq-from } M \ hs \longleftrightarrow (\exists M'. \text{valid-happ-seq } M \ hs \ M' \wedge M' \text{ }^c \models (\text{goal } P))$
 $\langle \text{proof} \rangle$

definition *mp-Fevl* :: (*object*, *rat*) *mapping* **where**
 $\text{mp-Fevl} = \text{Mapping.of-alist } (\text{fold } (\lambda a \text{ fev. case } a \text{ of } \text{Atom } (eqAtm \ f \ (\text{TimeEnt } t)) \Rightarrow (f, t) \# \text{fev} \mid - \Rightarrow \text{fev}) \ (\text{init } P) \ [])$

lemma *mp-func-eval-correct[simp]*: $\text{Mapping.lookup } \text{mp-Fevl} = \text{func-eval}$
 $\langle \text{proof} \rangle$

fun *duration-matches'* :: (*object*, *rat*) *mapping* $\Rightarrow$ *time* $\Rightarrow$ *term duration-constraint*
 $\Rightarrow$ (*variable* $\times$ *type*) *list* $\Rightarrow$ *object list* $\Rightarrow$ *bool* **where**
duration-matches' $\text{mp-fe } d \ \text{No-Const } \text{params } \text{args} \longleftrightarrow d \geq 0$
 $| \text{duration-matches'} \text{mp-fe } d \ (\text{Time-Const } op \ d') \ \text{params } \text{args} \longleftrightarrow$
 $(\text{case } op \text{ of}$
 $\quad \text{LEQ} \Rightarrow d \leq d'$
 $\quad | \text{EQ} \Rightarrow d = d')$

```

| GEQ  $\Rightarrow d \geq d'$ )
| duration-matches' mp-fe d (Func-Const op f vs) params args  $\longleftrightarrow$ 
  (let tsubst = subst-term (the o (map-of (zip (map fst params) args))) in
   case Mapping.lookup mp-fe (FuncEnt f (map tsubst vs)) of
     None  $\Rightarrow$  False
   | Some d'  $\Rightarrow$  (case op of
     LEQ  $\Rightarrow d \leq d'$ 
   | EQ  $\Rightarrow d = d'$ 
   | GEQ  $\Rightarrow d \geq d'$ ))

```

lemma (in wf-ast-problem) duration-matches'-correct:
duration-matches' mp-Fevl d dc params args $\longleftrightarrow$ duration-matches d dc params
args
⟨proof⟩

definition durations-match' mp-fe d xs params args = list-all (λx . duration-matches'
mp-fe d x params args) xs

lemma (in wf-ast-problem) durations-match'-correct:
durations-match' mp-Fevl d dcs params args $\longleftrightarrow$ durations-match d dcs params
args
⟨proof⟩

fun consec-https-in-interval-exec :: time list $\Rightarrow$ time $\Rightarrow$ time $\Rightarrow$ (time $\times$ time) list
where

```

consec-https-in-interval-exec (t#t'#ts) ti tj =
  (if t < ti then consec-https-in-interval-exec (t'#ts) ti tj
   else if ti ≤ t ∧ t' ≤ tj then (t,t')#(consec-https-in-interval-exec (t'#ts) ti tj)
   else [])
| consec-https-in-interval-exec ts ti tj = []

```

lemma consec-https-in-interval-exec-correct:
assumes strict-sorted ts
shows consec-https-in-interval ts t_i t_j = consec-https-in-interval-exec ts t_i t_j
⟨proof⟩

fun place-inv-snap-acts :: time list $\Rightarrow$ time $\Rightarrow$ time $\Rightarrow$ ground-action $\Rightarrow$ (time $\times$
ground-action) list **where**

```

place-inv-snap-acts (t#t'#ts) ti tj ainv =
  (if t < ti then place-inv-snap-acts (t'#ts) ti tj ainv
   else if ti ≤ t ∧ t' ≤ tj then ((t+t') div 2, ainv)#(place-inv-snap-acts (t'#ts) ti
tj ainv)
   else [])
| place-inv-snap-acts ts ti tj ainv = []

```

lemma place-inv-snap-acts-correct:
assumes strict-sorted ts
shows place-inv-snap-acts ts t₁ t₂ a_{inv} = map ($\lambda(t_i, t_j).$ ((t_i+t_j) div 2, a_{inv}))

(consec-https-in-interval ts t_1 t_2)
 ⟨proof⟩

Lift *simplify-action* into an Error Monad.

```

fun simplify-actionE :: - => (object, type) mapping => (object, rat) mapping =>
time list => (time × plan-action) => -(time × ground-action) list where
  simplify-actionE stg mp mp-fe https (tπ, Simple-Plan-Action n args) = do {
    check (tπ ≥ 0) (ERRS "Invalid starting time point");
    a ← resolve-action-schemaE n;
    check (case a of Simple-Action-Schema - - - => True | - => False) (ERRS
"Invalid Plan action type");
    check (action-params-match2 stg mp a args) (ERRS "Parameter mismatch");
    Error-Monad.return [(tπ, instantiate-action-schema a args At-Start)]
  }
  | simplify-actionE stg mp mp-fe https (tπ, Durative-Plan-Action n args d) = do {
    check (tπ ≥ 0) (ERRS "Invalid starting time point");
    a ← resolve-action-schemaE n;
    check (case a of Durative-Action-Schema - - - - => True | - => False) (ERRS
"Invalid Plan action type");
    check (action-params-match2 stg mp a args) (ERRS "Parameter mismatch");
    check (d ≥ 0) (ERRS "Invalid duration (negative)");
    check (durations-match' mp-fe d (duration-constraint a) (parameters a) args)
(ERRS "Duration constraint not satisfied");
    let ainv = inst-snap-action a args Over-All in
    Error-Monad.return (
      (tπ, inst-snap-action a args At-Start) #
      (tπ+d, inst-snap-action a args At-End) #
      (place-inv-snap-acts https tπ (tπ+d) ainv)
    )
  }

```

Justification of refinement for *simplify-action*.

lemma (in wf-ast-problem) *simplify-actionE-return-iff*:
assumes wf-domain **and** strict-sorted https
shows simplify-actionE STG mp-objT mp-Fevl https (t_π, π) = Inr as
 ⟷ (wf-plan-action π ∧ t_π ≥ 0 ∧ simplify-action https (t_π, π) = as)
 ⟨proof⟩

Lift *simplify-plan* into an Error Monad.

```

fun simplify-planE :: - => (object, type) mapping => (object, rat) mapping => time
list => plan => -+happening list where
  simplify-planE stg mp mp-fe https [] = do {
    Error-Monad.return []
  }
  | simplify-planE stg mp mp-fe https (π#πs) = do {
    as ← simplify-actionE stg mp mp-fe https π;
    hs ← simplify-planE stg mp mp-fe https πs;
    Error-Monad.return (insort-timed-ground-acts as hs)
  }

```

Justification of refinement for *simplify-plan*

lemma (in *wf-ast-problem*) *simplify-planE-return-iff*[*return-iff*]:
assumes *wf-domain* **and** *strict-sorted htps*
shows *simplify-planE STG mp-objT mp-Fevl htps* $\pi s = \text{Inr } hs$
 $\longleftrightarrow (\text{simplify-plan } htps \pi s = hs \wedge \text{wf-plan } \pi s)$
<proof>

type-synonym 'a tree-ht = ('a $\times$ nat) tree

fun *avl-height* :: *happening tree-ht* $\Rightarrow$ nat **where**
avl-height $\langle \rangle = 0$
| *avl-height* $\langle l, ((t, as), ht), r \rangle = ht$

definition *avl-node* :: *happening tree-ht* $\Rightarrow$ *happening* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
avl-node *l h r* = $\langle l, (h, (\max (\text{avl-height } l) (\text{avl-height } r) + 1)), r \rangle$

definition *avl-balL* :: *happening tree-ht* $\Rightarrow$ *happening* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
avl-balL *l a r* =
(if *avl-height* *l* = *avl-height* *r* + 2 then
case *l* of $\langle bl, (b, -), br \rangle \Rightarrow$
if *avl-height* *bl* < *avl-height* *br* then
case *br* of $\langle cl, (c, -), cr \rangle \Rightarrow \text{avl-node } (\text{avl-node } bl \ b \ cl) \ c \ (\text{avl-node } cr \ a \ r)$
else *avl-node* *bl* *b* (*avl-node* *br* *a* *r*)
else *avl-node* *l* *a* *r*)

definition *avl-balR* :: *happening tree-ht* $\Rightarrow$ *happening* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
avl-balR *l a r* =
(if *avl-height* *r* = *avl-height* *l* + 2 then
case *r* of $\langle bl, (b, -), br \rangle \Rightarrow$
if *avl-height* *bl* > *avl-height* *br* then
case *bl* of $\langle cl, (c, -), cr \rangle \Rightarrow \text{avl-node } (\text{avl-node } l \ a \ cl) \ c \ (\text{avl-node } cr \ b \ br)$
else *avl-node* (*avl-node* *l* *a* *bl*) *b* *br*
else *avl-node* *l* *a* *r*)

fun *insert-happ-to-tree* :: *happening* $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**

insert-happ-to-tree (*t_i*, *A_i*) $\langle \rangle = \text{avl-node } \langle \rangle \ (t_i, A_i) \ \langle \rangle$
| *insert-happ-to-tree* (*t_i*, *A_i*) $\langle l, ((t_j, A_j), h), r \rangle =$
if *t_i* < *t_j* then *avl-balL* (*insert-happ-to-tree* (*t_i*, *A_i*) *l*) (*t_j*, *A_j*) *r*
else if *t_i* = *t_j* then *avl-node* *l* (*t_j*, *A_i* @ *A_j*) *r*
else *avl-balR* *l* (*t_j*, *A_j*) (*insert-happ-to-tree* (*t_i*, *A_i*) *r*)

fun *insert-timed-ground-acts-to-tree* :: (time $\times$ ground-action) list $\Rightarrow$ *happening tree-ht* $\Rightarrow$ *happening tree-ht* **where**
insert-timed-ground-acts-to-tree [] *htree* = *htree*

| *insert-timed-ground-acts-to-tree* $((t_a, a) \# as)$ *htree* =
insert-happ-to-tree $(t_a, [a])$ (*insert-timed-ground-acts-to-tree* *as* *htree*)

definition *avl-inorder* :: *happening tree-ht* $\Rightarrow$ *happening list* **where**
avl-inorder *htree* = *map fst* (*inorder* *htree*)

fun *simplify-planE-avl'* :: $- \Rightarrow (\text{object}, \text{type}) \text{ mapping} \Rightarrow (\text{object}, \text{rat}) \text{ mapping} \Rightarrow$
time list $\Rightarrow$ *plan* $\Rightarrow$ $- + \text{happening tree-ht}$ **where**
simplify-planE-avl' *G mp mp-fe https []* = *do* {
Error-Monad.return $\langle \rangle$
}
| *simplify-planE-avl'* *G mp mp-fe https* $(\pi \# \pi s)$ = *do* {
as $\leftarrow$ *simplify-actionE* *G mp mp-fe https* π ;
htree $\leftarrow$ *simplify-planE-avl'* *G mp mp-fe https* πs ;
Error-Monad.return (*insert-timed-ground-acts-to-tree* *as* *htree*)
}

fun *simplify-planE-avl* :: $- \Rightarrow (\text{object}, \text{type}) \text{ mapping} \Rightarrow (\text{object}, \text{rat}) \text{ mapping} \Rightarrow$
time list $\Rightarrow$ *plan* $\Rightarrow$ $- + \text{happening list}$ **where**
simplify-planE-avl *G mp mp-fe https* πs = *do* {
htree $\leftarrow$ *simplify-planE-avl'* *G mp mp-fe https* πs ;
Error-Monad.return (*avl-inorder* *htree*)
}

lemma *insort-happ-consL*:
assumes $\forall (t_j, A_j) \in \text{set } hs_1. t_j < t_i$
shows $hs_1 @ (\text{insort-happ } (t_i, A_i) hs_2) = \text{insort-happ } (t_i, A_i) (hs_1 @ hs_2)$
 $\langle \text{proof} \rangle$

lemma *insort-happ-consR*:
assumes $\forall (t_j, A_j) \in \text{set } hs_2. t_i < t_j$
shows $(\text{insort-happ } (t_i, A_i) hs_1) @ hs_2 = \text{insort-happ } (t_i, A_i) (hs_1 @ hs_2)$
 $\langle \text{proof} \rangle$

lemma *inorder-avl-balL*: *avl-inorder* (*avl-balL* *l a r*) = *avl-inorder* *l* @ *a* #
avl-inorder *r*
 $\langle \text{proof} \rangle$

lemma *inorder-avl-balR*: *avl-inorder* (*avl-balR* *l a r*) = *avl-inorder* *l* @ *a* #
avl-inorder *r*
 $\langle \text{proof} \rangle$

lemma *strict-sorted-appl*: *strict-sorted* (*a* @ [*y*]) $\Longrightarrow \forall x \in \text{set } a. x < y$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-consg*: *strict-sorted* (*y* # *a*) $\Longrightarrow \forall x \in \text{set } a. y < x$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-avl-inorder-l:*

assumes *strict-sorted* (map fst (avl-inorder $\langle l, ((t_i, A_i), h), r \rangle$))

shows $\forall (t_j, A_j) \in \text{set } (\text{avl-inorder } l). t_j < t_i$

$\langle \text{proof} \rangle$

lemma *strict-sorted-avl-inorder-r:*

assumes *strict-sorted* (map fst (avl-inorder $\langle l, ((t_i, A_i), h), r \rangle$))

shows $\forall (t_j, A_j) \in \text{set } (\text{avl-inorder } r). t_i < t_j$

$\langle \text{proof} \rangle$

lemma *insert-happ-to-tree-set:*

assumes *strict-sorted* (map fst (avl-inorder htree))

shows $\text{set } (\text{map fst } (\text{avl-inorder } (\text{insert-happ-to-tree } (t_i, A_i) \text{ htree}))) = \text{set } (\text{map fst } (\text{avl-inorder htree})) \cup \{t_i\}$

$\langle \text{proof} \rangle$

lemma *insert-timed-ground-act-to-tree-strict-sorted:*

assumes *strict-sorted* (map fst (avl-inorder htree))

shows *strict-sorted* (map fst (avl-inorder (insert-happ-to-tree h htree)))

$\langle \text{proof} \rangle$

lemma *insert-timed-ground-acts-to-tree-strict-sorted:*

assumes *strict-sorted* (map fst (avl-inorder htree))

shows *strict-sorted* (map fst (avl-inorder (insert-timed-ground-acts-to-tree as htree)))

$\langle \text{proof} \rangle$

lemma *insert-happ-to-tree-equiv:*

assumes *strict-sorted* (map fst (avl-inorder htree))

shows $\text{avl-inorder } (\text{insert-happ-to-tree } (t_i, A_i) \text{ htree}) = \text{insort-happ } (t_i, A_i) (\text{avl-inorder htree})$

$\langle \text{proof} \rangle$

lemma *insert-timed-ground-acts-to-tree-equiv:*

assumes *strict-sorted* (map fst (avl-inorder htree))

shows *insort-timed-ground-acts as* (avl-inorder htree)

$= \text{avl-inorder } (\text{insert-timed-ground-acts-to-tree as htree})$

$\langle \text{proof} \rangle$

lemma *simplify-planE-avl'-strict-sorted:*

assumes *simplify-planE-avl'* STG mp-objT mp-Fevl https $\pi s = \text{Inr htree}$

shows *strict-sorted* (map fst (avl-inorder htree))

$\langle \text{proof} \rangle$

lemma *simplify-planE-avl'-equiv:*

assumes *simplify-planE* STG mp-objT mp-Fevl https $\pi s = \text{Inr hs}$

and *simplify-planE-avl'* STG mp-objT mp-Fevl https $\pi s = \text{Inr htree}$

shows $hs = \text{avl-inorder } htree$
 $\langle \text{proof} \rangle$

lemma *simplify-planE-avl-equiv-Inr*:

shows $(\exists hs. \text{simplify-planE } STG \text{ mp-objT mp-Fevl htps } \pi s = \text{Inr } hs)$
 $\longleftrightarrow (\exists htree. \text{simplify-planE-avl}' STG \text{ mp-objT mp-Fevl htps } \pi s = \text{Inr } htree)$
 $\langle \text{proof} \rangle$

lemma *simplify-planE-avl-equiv*:

shows $\text{simplify-planE } STG \text{ mp-objT mp-Fevl htps } \pi s = \text{Inr } hs$
 $\longleftrightarrow \text{simplify-planE-avl } STG \text{ mp-objT mp-Fevl htps } \pi s = \text{Inr } hs$
 $\langle \text{proof} \rangle$

lemma (in *wf-ast-problem*)

assumes *wf-domain*

and $\text{simplify-planE-avl } STG \text{ mp-objT mp-Fevl (htps-exec } \pi s) \pi s = \text{Inr } hs$

shows $\text{ind-happ-seq } \pi s \text{ } hs$ **and** $\text{wf-plan } \pi s$

$\langle \text{proof} \rangle$

Lift *valid-happ-seq-from* into an Error Monad, and use efficient happening execution *en-exE*

fun *valid-happ-seq-fromE* :: $\text{nat} \Rightarrow \text{world-model} \Rightarrow \text{happening list} \Rightarrow \text{-+unit}$
where
 $\text{valid-happ-seq-fromE } si \ s \ [] =$
 $\text{check (holds } s \ (\text{goal } P)) \ (\text{ERRS "Postcondition does not hold"})$
 $| \text{valid-happ-seq-fromE } si \ s \ (h\#hs) = \text{do } \{$
 $\quad s \leftarrow \text{en-exE } h \ s \ <+? \ (\lambda e \ -. \ \text{shows "at step " } o \ \text{shows } si \ o \ \text{shows "": " } o \ e \ ());$
 $\quad \text{valid-happ-seq-fromE } (si+1) \ s \ hs$
 $\}$

For the refinement, we need to show that the world models only contain atoms, i.e., containing only atoms is an invariant under execution of well-formed happenings.

lemma (in *wf-ast-problem*) *wf-happ-only-add-atoms*:

assumes *wm-basic s*

and $\forall a \in \text{set } as. \text{wf-ground-action } a$

shows $\text{wm-basic } (\text{apply-happ-exec } (t, as) \ s)$

$\langle \text{proof} \rangle$

lemma (in *wf-ast-problem*) *wf-happ-only-add-atoms'*:

assumes *wf-world-model s*

and $\forall a \in \text{set } as. \text{wf-ground-action } a$

shows $\text{wf-world-model } (\text{apply-happ-exec } (t, as) \ s)$

$\langle \text{proof} \rangle$

Justification for refinement of *valid-happ-seq-from*.

lemma (in *wf-ast-problem*) *valid-happ-seq-fromE-return-iff*[*return-iff*]:

assumes *wf-basic s*
and $\forall (t, as) \in \text{set } hs. \forall a \in \text{set } as. \text{wf-ground-action } a$
shows $\text{valid-happ-seq-from } s \ hs \longleftrightarrow \text{valid-happ-seq-fromE } k \ s \ hs = \text{Inr } ()$
 $\langle \text{proof} \rangle$

Now we can define a refinement for *valid-plan-from2*.

fun *valid-plan-from2-exec* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *bool* **where**
valid-plan-from2-exec *s* $\pi s \longleftrightarrow$
 $(\text{wf-plan } \pi s \wedge \text{valid-happ-seq-from } s \ (\text{simplify-plan } (\text{https-exec } \pi s) \ \pi s))$

With $\llbracket \text{wf-ast-problem } ?P; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs; \text{ast-problem.ind-happ-seq } ?P \ ?\pi s \ ?hs'; \text{ast-problem.wf-plan } ?P \ ?\pi s; \text{valid-happ-seq } ?M_0 \ ?hs \ ?M_n \rrbracket \implies \text{valid-happ-seq } ?M_0 \ ?hs' \ ?M_n$ we can prove, that if there exists a valid induced happening sequence, then simplify plan is also valid.

lemma (**in** *wf-ast-problem*) *simplify-plan-valid-exists1*:
fixes πs
defines $\text{https} \equiv \text{https-exec } \pi s$
assumes *wf-plan* πs
and $\exists hs. \text{ind-happ-seq } \pi s \ hs \wedge \text{valid-happ-seq } M \ hs \ M' \wedge M' \ c \models = (\text{goal } P)$
shows $\text{valid-happ-seq } M \ (\text{simplify-plan } \text{https } \pi s) \ M' \wedge M' \ c \models = (\text{goal } P)$
 $\langle \text{proof} \rangle$

Other way round is easy, since *simplify-plan* is an induced happening sequence.

lemma (**in** *wf-ast-problem*) *simplify-plan-valid-exists2*:
fixes πs
defines $\text{https} \equiv \text{https-exec } \pi s$
assumes *wf-plan* πs
and $\text{valid-happ-seq } M \ (\text{simplify-plan } \text{https } \pi s) \ M' \wedge M' \ c \models = (\text{goal } P)$
shows $\exists hs. \text{ind-happ-seq } \pi s \ hs \wedge \text{valid-happ-seq } M \ hs \ M' \wedge M' \ c \models = (\text{goal } P)$
 $\langle \text{proof} \rangle$

Justification for the refinement of *valid-plan-from2*.

lemma (**in** *wf-ast-problem*) *valid-plan-from2-exec-refine*:
 $\text{valid-plan-from2 } s \ \pi s \longleftrightarrow \text{valid-plan-from2-exec } s \ \pi s$
 $\langle \text{proof} \rangle$

Now we can also lift *valid-plan-from2-exec* into an Error Monad, using *simplify-planE* and *valid-happ-seq-fromE*

fun *valid-plan-from2E* :: $- \Rightarrow (\text{object}, \text{type}) \text{ mapping} \Rightarrow (\text{object}, \text{rat}) \text{ mapping} \Rightarrow$
world-model $\Rightarrow$ *plan* $\Rightarrow$ $- + \text{unit}$ **where**
valid-plan-from2E *stg* *mp* *mp-fe* *s* $\pi s = \text{do } \{$
 $hs \leftarrow \text{simplify-planE-avl } \text{stg } \text{mp } \text{mp-fe} \ (\text{https-exec } \pi s) \ \pi s;$
 $\text{valid-happ-seq-fromE } 1 \ s \ hs$
 $\}$

lemma (in *wf-ast-problem*) *valid-plan-fromE-wf-plan*:
assumes *valid-plan-from2E STG mp-objT mp-Fevl s π s = Inr ()*
shows *wf-plan π s*
<proof>

Refinement lemma for our plan checking algorithm

lemma (in *wf-ast-problem*) *valid-plan-from2E-return-iff*:
assumes *wf-world-model s*
shows *valid-plan-from2E STG mp-objT mp-Fevl s π s = Inr ()* $\longleftrightarrow$ *valid-plan-from2E s π s*
<proof>

lemma (in *wf-ast-problem*)
assumes *wf-world-model s*
shows *valid-plan-from2E STG mp-objT mp-Fevl s π s = Inr ()* $\longleftrightarrow$ *valid-plan-from2E s π s*
<proof>

lemmas *valid-plan-from2E-return-iff'[return-iff]*
 $=$ *wf-ast-problem.valid-plan-from2E-return-iff[of P, OF wf-ast-problem.intro]*

end — Context of *ast-problem*

2.3 Executable Plan Checker

We obtain the main plan checker by combining the well-formedness check and executability check.

definition *check-all-list P l msg msgf* $\equiv$
forallM (λx. check (P x) (λ::unit. shows msg o shows " : " o msgf x)) l <+?
snd

lemma *check-all-list-return-iff'[return-iff]*: *check-all-list P l msg msgf = Inr ()* $\longleftrightarrow$
 $(\forall x \in \text{set } l. P x)$
<proof>

definition *check-wf-types D* $\equiv$ *do {*
check-all-list (λ(-,t). t=(STR "object") $\vee$ $t \in \text{fst} \text{'set (types D)}$) (types D) "Undeclared
supertype" (shows o snd)
}

lemma *check-wf-types-return-iff*[*return-iff*]: *check-wf-types D = Inr ()* $\longleftrightarrow$ *ast-domain.wf-types D*

<proof>

definition *check-wf-domain D stg conT* $\equiv$ *do* {
check-wf-types D;
check (distinct (map (predicate-decl.pred) (predicates D))) (ERRS "Duplicate predicate declaration");
check-all-list (ast-domain.wf-predicate-decl D) (predicates D) "Malformed predicate declaration" (*shows o predicate.name o predicate-decl.pred*);
check (distinct (map (function-decl.func) (functions D))) (ERRS "Duplicate function declaration");
check-all-list (ast-domain.wf-function-decl D) (functions D) "Malformed function declaration" (*shows o func.name o function-decl.func*);
check (distinct (map fst (consts D))) (ERRS "Duplicate constant declaration");
*check (($\forall (n, T) \in \text{set } (\text{consts } D).$ *ast-domain.wf-type D T*) (ERRS "Malformed type"))*;
check (distinct (map ast-action-schema.name (actions D))) (ERRS "Duplicate action name");
check-all-list (ast-domain.wf-action-schema' D stg conT) (actions D) "Malformed action" (*shows o ast-action-schema.name*)
}

lemma *check-wf-domain-return-iff*[*return-iff*]:

check-wf-domain D stg conT = Inr () $\longleftrightarrow$ *ast-domain.wf-domain' D stg conT*

<proof>

definition *prepend-err-msg msg e* $\equiv$ $\lambda :: \text{unit}. \text{shows msg o shows "": " o e ()}$

definition *check-wf-problem P stg conT mp* $\equiv$ *do* {
let D = ast-problem.domain P;
check-wf-domain D stg conT $<+ ?$ *prepend-err-msg "Domain not well-formed"*;
check (distinct (map fst (objects P) @ map fst (consts D))) (ERRS "Duplicate object declaration");
*check (($\forall (n, T) \in \text{set } (\text{objects } P).$ *ast-domain.wf-type D T*) (ERRS "Malformed type"))*;
check (distinct (init P)) (ERRS "Duplicate fact in initial state");
*check (($\forall f \in \text{set } (\text{init } P).$ *ast-problem.wf-fmla-atom2' P mp stg f* $\vee$ *ast-problem.wf-func-assign' P mp stg f*) (ERRS "Malformed formula in initial state"))*;
check (ast-domain.wf-fmla' D (Mapping.lookup mp) stg (goal P)) (ERRS "Malformed goal formula")
}

lemma *check-wf-problem-return-iff*[*return-iff*]:

check-wf-problem P stg conT mp = Inr () $\longleftrightarrow$ *ast-problem.wf-problem' P stg conT mp*

<proof>

definition *check-plan P π s* $\equiv$ *do* {

```

let stg = ast-domain.STG (ast-problem.domain P);
let conT = ast-domain.mp-constT (ast-problem.domain P);
let mp = ast-problem.mp-objT P;
let mp-fe = ast-problem.mp-Fevl P;
check-wf-problem P stg conT mp;
ast-problem.valid-plan-from2E P stg mp mp-fe (ast-problem.I P) πs
} <+? (λe. String.implode (e () ""))

```

Correctness theorem of the plan checker: It returns *Inr* () if and only if the problem is well-formed and the plan is valid.

theorem *check-plan-return-iff*[*return-iff*]:
check-plan P πs = Inr () $\longleftrightarrow$ *ast-problem.wf-problem P* $\wedge$ *ast-problem.valid-plan2 P πs*
<proof>

find-theorems *ast-problem.valid-plan-from2*

2.4 Code Setup

In this section, we set up the code generator to generate verified code for our plan checker.

2.4.1 Code Equations

We first register the code equations for the functions of the checker. Note that we not necessarily register the original code equations, but also optimized ones.

lemmas *wf-domain-code* =
ast-domain.sig-def
ast-domain.wf-types-def
ast-domain.wf-type.simps
ast-domain.wf-predicate-decl.simps
ast-domain.STG-def
ast-domain.is-of-type'-def
ast-domain.wf-atom'.simps
ast-domain.wf-pred-atom'.simps
ast-domain.wf-fmla'.simps
ast-domain.wf-fmla-atom1'.simps
ast-domain.wf-duration-consts'-def
ast-domain.wf-effect'.simps
ast-domain.wf-action-schema'.simps
ast-domain.wf-domain'-def
ast-domain.subst-term.simps
ast-domain.mp-constT-def

ast-domain.wf-function-decl.simps
ast-domain.func-sig-def

```

ast-domain.wf-func-args'.simps
ast-domain.wf-duration-const'.simps
ast-domain.acts-non-intrf-def

```

```
declare wf-domain-code[code]
```

```

lemmas wf-problem-code =
  ast-problem.durations-match'-def
  ast-problem.wf-problem'-def
  ast-problem.wf-fact'-def
  ast-problem.is-obj-of-type-alt
  ast-problem.wf-fact-def
  ast-problem.wf-plan-action.simps
  ast-domain.subtype-edge.simps

```

```
declare wf-problem-code[code]
```

```

lemmas check-code =
  ast-problem.valid-plan2-def
  ast-problem.valid-plan-from2E.simps
  ast-problem.res-inst.simps
  ast-domain.resolve-action-schema-def
  ast-domain.resolve-action-schemaE-def
  ast-problem.I-def
  ast-domain.instantiate-action-schema.simps
  ast-problem.holds-def
  ast-problem.mp-objT-def
  ast-problem.is-obj-of-type-impl-def
  ast-problem.wf-fmla-atom2'-def
  valuation-def

```

```

ast-problem.valid-happ-seq-fromE.simps
ast-problem.htps-exec.simps
ast-problem.insort-htp.simps
ast-problem.simplify-planE.simps
ast-problem.simplify-actionE.simps
ast-problem.insort-happ.simps
ast-domain.inst-snap-action.simps
ast-domain.filter-time-spec-def
ast-domain.conjunct-effects-def

```

```

ast-problem.apply-happ-exec.simps
ast-problem.en-exE-def
ast-problem.insort-mult-happs.simps
ast-problem.duration-matches'.simps
ast-problem.wf-func-assign'.simps

```

```
ast-problem.mp-Fevl-def
```

ast-problem.place-inv-snap-acts.simps
ast-problem.insort-timed-ground-acts.simps
ast-problem.insert-happ-to-tree.simps
ast-problem.insert-timed-ground-acts-to-tree.simps
ast-problem.avl-inorder-def
ast-problem.simplify-planE-avl.simps
ast-problem.simplify-planE-avl'.simps
ast-problem.avl-height.simps
ast-problem.avl-node-def
ast-problem.avl-balR-def
ast-problem.avl-balL-def

declare *check-code*[*code*]

2.4.2 Setup for Containers Framework

derive (*linorder*) *compare rat*

derive (*eq*) *ceq predicate func atom object formula*
derive *ccompare predicate func atom object formula*
derive (*no*) *cenum atom object formula*
derive (*rbt*) *set-impl func atom object formula*

derive (*rbt*) *mapping-impl object*

derive *linorder predicate func object atom object atom formula*

2.4.3 More Efficient Distinctness Check for Linorders

fun *no-stutter* :: 'a list $\Rightarrow$ bool **where**
no-stutter [] = True
| *no-stutter* [-] = True
| *no-stutter* (a#b#l) = (a#b $\wedge$ *no-stutter* (b#l))

lemma *sorted-no-stutter-eq-distinct*: *sorted l $\implies$ no-stutter l $\longleftrightarrow$ distinct l*
<proof>

definition *distinct-ds* :: 'a::linorder list $\Rightarrow$ bool
where *distinct-ds l $\equiv$ no-stutter (quicksort l)*

lemma [*code-unfold*]: *distinct = distinct-ds*
<proof>

2.4.4 Parsing Rational Numbers

type-synonym *digit* = nat

Well-formedness conditions for a digit and a sequence of digits.

definition $wf_digit\ d \longleftrightarrow d < 10$

A sequence of digits is well-formed iff it is normalized and only contains well-formed digits.

definition $wf_digits\ ds \longleftrightarrow (\forall d \in set\ ds. wf_digit\ d)$

Functions to trim leading and trailing zeros.

fun $trim_ld_zs :: digit\ list \Rightarrow digit\ list$ **where**
 $trim_ld_zs\ [] = []$
 $| trim_ld_zs\ (d\#\ ds) =$
 $(if\ d = 0\ then\ trim_ld_zs\ ds\ else\ d\#\ ds)$

lemma $tlz_hd_neg_z$: $trim_ld_zs\ ds = ds' \implies hd\ ds' \neq 0 \vee ds' = []$
 $\langle proof \rangle$

lemma $trim_ld_zs_idem$: $trim_ld_zs\ (trim_ld_zs\ ds) = trim_ld_zs\ ds$
 $\langle proof \rangle$

lemma $trim_ld_zs_wf$: $wf_digits\ ds \implies wf_digits\ (trim_ld_zs\ ds)$
 $\langle proof \rangle$

lemma $trim_ld_zs_app$: $trim_ld_zs\ (ds1\ @\ ds2) = (trim_ld_zs\ ds1)\ @\ ds2 \vee (\forall d \in set\ ds1. d = 0)$
 $\langle proof \rangle$

fun $trim_tr_zs' :: digit\ list \Rightarrow digit\ list \Rightarrow digit\ list$ **where**
 $trim_tr_zs'\ []\ zs = []$
 $| trim_tr_zs'\ (d\#\ ds)\ zs =$
 $(if\ d = 0\ then\ trim_tr_zs'\ ds\ (0\#\ zs)$
 $else\ zs\ @\ d\ \# (trim_tr_zs'\ ds\ []))$

fun $trim_tr_zs :: digit\ list \Rightarrow digit\ list$ **where**
 $trim_tr_zs\ ds = trim_tr_zs'\ ds\ []$

lemma $tlz_last_neg_z_aux$:
assumes $trim_tr_zs'\ ds\ zs = ds'$
shows $last\ ds' \neq 0 \vee ds' = []$
 $\langle proof \rangle$

lemma $tlz_last_neg_z$: $trim_tr_zs\ ds = ds' \implies last\ ds' \neq 0 \vee ds' = []$
 $\langle proof \rangle$

lemma $trim_tr_zs'_wf$: $wf_digits\ ds \wedge wf_digits\ zs \implies wf_digits\ (trim_tr_zs'\ ds\ zs)$
 $\langle proof \rangle$

lemma $trim_tr_zs_wf$: $wf_digits\ ds \implies wf_digits\ (trim_tr_zs\ ds)$
 $\langle proof \rangle$

Functions to convert between integers and a sequence of digits.

```

fun int-of-digits :: digit list  $\Rightarrow$  int  $\Rightarrow$  int where
  int-of-digits [] acc = acc
| int-of-digits (d#ds) acc = int-of-digits ds (acc * 10 + d)

```

```

fun digits-of-int :: int  $\Rightarrow$  digit list where
  digits-of-int i =
    (if i  $\leq$  0 then []
     else if i < 10 then [nat i]
     else digits-of-int (i div 10) @ [nat (i mod 10)])

```

```

value digits-of-int (int-of-digits [1,2,3,4] 0)
value int-of-digits (digits-of-int 1234) 0

```

```

value digits-of-int (int-of-digits [1] 123)

```

```

lemma digits-of-int-int-of-digits-aux:
  assumes acc > 0 and  $\forall d \in \text{set } ds. \text{wf-digit } d$ 
  shows digits-of-int (int-of-digits ds acc) = digits-of-int acc @ ds
  <proof>

```

Proof for correctness of function *int-of-digits* and *digits-of-int*.

```

lemma digits-of-int-int-of-digits:
  assumes wf-digits ds
  shows digits-of-int (int-of-digits ds 0) = trim-ld-zs ds
  <proof>

```

Function to convert from a sequences of digits to a rational number.

```

primrec rat-of-digits-pair :: digit list  $\times$  digit list  $\Rightarrow$  rat where
  rat-of-digits-pair (ds1,ds2) = (
    let ds1' = trim-ld-zs ds1; ds2' = trim-tr-zs ds2 in
    Rat.Fract (int-of-digits (ds1' @ ds2') 0) (10 ^ length ds2')
  )

```

For proofs about the function *rat-of-digits-pair* see `Rat_Parsing.thy`.

2.4.5 Code Generation

```

export-code
  check-plan
  nat-of-integer integer-of-nat int-of-integer integer-of-int Inl Inr Rat.Fract Rat.of-int
rat-of-digits-pair
  predAtm eqAtm predicate Pred Func Either Var Obj PredDecl FuncDecl BigAnd
BigOr
  formula.Not formula.Bot Effect No-Const Time-Const Func-Const LEQ EQ GEQ
Simple-Action-Schema Durative-Action-Schema At-Start At-End Over-All
map-atom Domain Problem Simple-Plan-Action Durative-Plan-Action
  term.CONST term.VAR
  String.explode String.implode
in SML

```

module-name *TEMPORAL-PDDL-Checker-Exported*
file *code/TEMPORAL-PDDL-Checker-Exported.sml*

end — Theory

3 Connecting the Semantics to State-Transition Based Semantics

theory *TEMPORAL-PDDL-Semantics-Alt*
imports
 TEMPORAL-PDDL-Semantics
 TEMPORAL-PDDL-Checker
begin

Auxiliary lemma with about *dropWhile*, *takeWhile* with *le* and *leq*

lemma *strict-sorted-drop-le-take-leq-comm:*
 assumes $t_i \leq t_j$
 and *strict-sorted* (*map fst hs*)
 shows $\text{dropWhile } (le\ t_i) (\text{takeWhile } (leq\ t_j)\ hs) = \text{takeWhile } (leq\ t_j) (\text{dropWhile } (le\ t_i)\ hs)$
 $\langle proof \rangle$

lemma *strict-sorted-drop-leq-take-le-comm:*
 assumes $t_i < t_j$
 and *strict-sorted* (*map fst hs*)
 shows $\text{dropWhile } (leq\ t_i) (\text{takeWhile } (le\ t_j)\ hs) = \text{takeWhile } (le\ t_j) (\text{dropWhile } (leq\ t_i)\ hs)$
 $\langle proof \rangle$

lemma *strict-sorted-drop-leq-take-leq-comm:*
 assumes $t_i \leq t_j$
 and *strict-sorted* (*map fst hs*)
 shows $\text{dropWhile } (leq\ t_i) (\text{takeWhile } (leq\ t_j)\ hs) = \text{takeWhile } (leq\ t_j) (\text{dropWhile } (leq\ t_i)\ hs)$
 $\langle proof \rangle$

lemma *strict-sorted-drop-le-take-leq:*
 assumes *strict-sorted* (*map fst hs*)
 and $(t_a, A) \in \text{set } hs$
 shows $(t_a, A) \in \text{set } (\text{dropWhile } (le\ t_i) (\text{takeWhile } (leq\ t_j)\ hs)) \iff t_i \leq t_a \wedge t_a \leq t_j$
 $\langle proof \rangle$

lemma *strict-sorted-drop-leq-take-leq*:
assumes *strict-sorted* (*map fst hs*)
and $(t_a, A) \in \text{set } hs$
shows $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) hs)) \longleftrightarrow t_i < t_a \wedge t_a \leq t_j$
 $\langle \text{proof} \rangle$

lemma *strict-sorted-drop-leq-take-le*:
assumes *strict-sorted* (*map fst hs*)
and $(t_a, A) \in \text{set } hs$
shows $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)) \longleftrightarrow t_i < t_a \wedge t_a < t_j$
 $\langle \text{proof} \rangle$

lemma *split-list2-le*:
assumes *strict-sorted* *xs*
and $x_1 \in \text{set } xs$
and $x_2 \in \text{set } xs$
shows $x_1 < x_2 \longleftrightarrow (\exists xs_1 xs_2 xs_3. xs = xs_1 @ x_1 \# xs_2 @ x_2 \# xs_3)$
 $\langle \text{proof} \rangle$

lemma *drop-take-leq_i-leq-leq_j-aux*:
assumes $t_k \leq t_j$
and *strict-sorted* (*map fst hs*)
shows $\text{takeWhile } (\text{leq } t_k) hs @ \text{takeWhile } (\text{leq } t_j) (\text{dropWhile } (\text{leq } t_k) hs) = \text{takeWhile } (\text{leq } t_j) hs$
 $\langle \text{proof} \rangle$

lemma *drop-take-leq_i-leq-leq_j-simp*:
assumes $t_i \leq t_k$ **and** $t_k \leq t_j$
and *strict-sorted* (*map fst hs*)
shows $\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_k) hs) @ \text{dropWhile } (\text{leq } t_k) (\text{takeWhile } (\text{leq } t_j) hs) = \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) hs)$
 $\langle \text{proof} \rangle$

3.1 State-Transition Based Semantics

context *ast-problem* **begin**

fun *inst-cond* :: *ast-action-schema* $\Rightarrow$ *object list* $\Rightarrow$ *temporal-annotation* $\Rightarrow$ (*object atom*) *formula* **where**

inst-cond (*Durative-Action-Schema* *n* *params* *d* *cond* *eff*) *args* *ts* =
 (let *tsubst* = *subst-term* (*the o* (*map-of* (*zip* (*map fst params*) *args*))) in
 (*map-formula o map-atom*) *tsubst* (*BigAnd* (*filter-time-spec* *ts cond*)))

fun *res-inst-inv* :: *plan-action* $\Rightarrow$ (*object atom*) *formula* *option* **where**
res-inst-inv (*Simple-Plan-Action* *n* *args*) = *None*

| *res-inst-inv* (*Durative-Plan-Action* n *args* d) =
 Some (*inst-cond* (*the* (*resolve-action-schema* n)) *args* *Over-All*)

lemma *res-inst-inv-refine*:

assumes *wf-plan-action* π
 and *Some* $a = \text{res-inst-snap-action } \pi \text{ Over-All}$
 shows *Some* (*precondition* a) = *res-inst-inv* π
 $\langle \text{proof} \rangle$

Set of all ground actions that, are executed at a given time point in the induced happening sequence.

definition *acts-of-plan-at* :: *time* $\Rightarrow$ *plan* $\Rightarrow$ *ground-action set* **where**
 acts-of-plan-at t_i πs =
 $\{a_\pi. \exists \pi. (t_i, \pi) \in \text{simple-acts } \pi s \wedge \text{Some } a_\pi = \text{res-inst } \pi \text{ At-Start}\}$
 $\cup \{a_{\text{start}}. \exists \pi. (t_i, \pi) \in \text{durative-acts } \pi s \wedge \text{Some } a_{\text{start}} = \text{res-inst-snap-action } \pi \text{ At-Start}\}$
 $\cup \{a_{\text{end}}. \exists t' \pi. (t', \pi) \in \text{durative-acts } \pi s \wedge t_i = t' + \text{duration } \pi \wedge \text{Some } a_{\text{end}} = \text{res-inst-snap-action } \pi \text{ At-End}\}$

Set of all invariant that have to hold in between consecutive happening time points $t' < t$.

definition *invs-of-plan-at* :: *time* $\Rightarrow$ *plan* $\Rightarrow$ (*object atom*) *formula set* **where**
 invs-of-plan-at t_i πs =
 $\{\text{inv}. \exists t_\pi \pi. (t_\pi, \pi) \in \text{durative-acts } \pi s \wedge t_\pi < t_i \wedge t_i \leq t_\pi + (\text{duration } \pi) \wedge \text{Some } \text{inv} = \text{res-inst-inv } \pi\}$

Predicate for sequence of happening time points.

fun *apply-eff* :: *ground-action set* $\Rightarrow$ *world-model* $\Rightarrow$ *world-model* **where**
 apply-eff A_i $M = (M - \bigcup (\text{set 'dels' effect 'A}_i)) \cup \bigcup (\text{set 'adds' effect 'A}_i)$

Instead of explicitly defining an induced happening sequence. Here the happening sequence is only implicitly constructed. Only happening time points are considered for the valid execution, invariants are treated like preconditions of an entire happening.

fun *valid-state-seq* :: *world-model* $\Rightarrow$ *time list* $\Rightarrow$ *plan* $\Rightarrow$ *world-model* $\Rightarrow$ *bool* **where**

valid-state-seq $M \ [] \ \pi s \ M' \longleftrightarrow (M = M')$
 | *valid-state-seq* $M \ (t_i \# ts) \ \pi s \ M' \longleftrightarrow$
 (*let* $A_i = \text{acts-of-plan-at } t_i \ \pi s$ *in*
 $(\forall i \in \text{invs-of-plan-at } t_i \ \pi s. M \stackrel{c}{\models} i)$
 $\wedge (\forall a \in A_i. M \stackrel{c}{\models} \text{precondition } a)$
 $\wedge (\forall a \in A_i. \forall b \in A_i. a \neq b \longrightarrow \text{acts-non-intrf } a \ b)$
 $\wedge \text{valid-state-seq } (\text{apply-eff } A_i \ M) \ ts \ \pi s \ M')$

definition *valid-plan-from* :: *world-model* $\Rightarrow$ *plan* $\Rightarrow$ *bool* **where**
 valid-plan-from $M \ \pi s \longleftrightarrow (\text{wf-plan } \pi s \wedge (\exists \text{https } M'. \text{https-seq } \pi s \ \text{https} \wedge \text{valid-state-seq } M \ \text{https } \pi s \ M' \wedge M' \stackrel{c}{\models} (\text{goal } P)))$

definition *valid-plan* :: *plan* $\Rightarrow$ *bool*
where *valid-plan* $\equiv$ *valid-plan-from I*

Concise definition used in paper:

lemma *valid-plan* $\pi s \equiv$ *wf-plan* $\pi s \wedge (\exists \text{https } M'. \text{https-seq } \pi s \text{ https} \wedge \text{valid-state-seq } I \text{ https } \pi s M' \wedge M' \models_{\text{c}} (\text{goal } P))$
 $\langle \text{proof} \rangle$

3.2 Connecting the State-Transition Based Semantics to Happening Sequence Based Semantics

Justification for Semantic refinement

lemma *apply-happ* $(t, A) M = \text{apply-eff } (\text{set } A) M$
 $\langle \text{proof} \rangle$

lemma *valid-state-seq-app-iff*:
 $\text{valid-state-seq } M_i (ts_1 @ ts_2) \pi s M_k$
 $\longleftrightarrow (\exists M_j. \text{valid-state-seq } M_i ts_1 \pi s M_j \wedge \text{valid-state-seq } M_j ts_2 \pi s M_k)$
 $\langle \text{proof} \rangle$

lemma *valid-state-seq-state-unique*:
assumes *valid-state-seq* $M_0 ts \pi s M_i$ **and** *valid-state-seq* $M_0 ts \pi s M_i'$
shows $M_i = M_i'$
 $\langle \text{proof} \rangle$

acts-of-plan-at returns exactly the set of actions that would be executed for a happening time point in the induced happening sequence

lemma *ind-happ-seq-htp-acts-of-plan-at*:
assumes *ind-happ-seq* $\pi s \text{ hs}$
and *is-htp* $\pi s t$
and $(t, A) \in \text{set } \text{hs}$
shows $\text{set } A = \text{acts-of-plan-at } t \pi s$
 $\langle \text{proof} \rangle$
end

context *wf-ast-problem*
begin

The precondition of every invariant ground action in between two consecutive happening time points is included in *invs-of-plan-at*.

lemma *ind-happ-seq-invs-of-plan-at*:
assumes *wf-plan* πs
and *consec-https* $\pi s t_i t_j$
and *ind-happ-seq* $\pi s \text{ hs}$
and $(t, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) \text{ hs}))$
and $a \in \text{set } A$

shows $(\text{precondition } a) \in \text{invs-of-plan-at } t_j \pi s$
 $\langle \text{proof} \rangle$

Every invariant from *invs-of-plan-at* is placed as an invariant ground action in between two consecutive happening time points in the induced happening sequence.

lemma *invs-of-plan-at-ind-happ-seq*:
assumes *wf-plan* πs
and *consec-htps* $\pi s t_i t_j$
and *ind-happ-seq* $\pi s hs$
and $i \in \text{invs-of-plan-at } t_j \pi s$
shows $\exists (t, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)). \exists a \in \text{set } A. i = \text{precondition } a$
 $\langle \text{proof} \rangle$

In an induced happening sequence all ground actions between two consecutive happening time points have empty effect. Those ground actions are the instantiated durative action for the (**over all** ...) time specifier.

lemma *ind-happ-seq-leq_i-le_j-empty-eff*:
assumes *consec-htps* $\pi s t_i t_j$
and *wf-plan* πs
and *ind-happ-seq* $\pi s hs$
and $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs))$
and $a \in \text{set } A$
shows $\text{effect } a = \text{Effect } [] []$
 $\langle \text{proof} \rangle$

Applying a happening containing only action with empty effect, doesn't change the current state.

lemma *apply-happ-empty-effect*:
assumes $\forall (t_a, A) \in \text{set } hs. \forall a \in \text{set } A. \text{effect } a = \text{Effect } [] []$
shows $\forall h \in \text{set } hs. \text{apply-happ } h M = M$
 $\langle \text{proof} \rangle$

Combining $\llbracket \text{wf-ast-problem } ?P; \text{consec-htps } ?\pi s ?t_i ?t_j; \text{ast-problem.wf-plan } ?P ?\pi s; \text{ast-problem.ind-happ-seq } ?P ?\pi s ?hs; (?t_a, ?A) \in \text{set } (\text{dropWhile } (\text{leq } ?t_i) (\text{takeWhile } (\text{le } ?t_j) ?hs)); ?a \in \text{set } ?A \rrbracket \implies \text{ground-action.effect } ?a = \text{Effect } [] []$ and $\forall (t_a, A) \in \text{set } ?hs. \forall a \in \text{set } A. \text{ground-action.effect } a = \text{Effect } [] [] \implies \forall h \in \text{set } ?hs. \text{apply-happ } h ?M = ?M$: Applying the happening in between two consecutive happening time points doesn't change the current state.

lemma *ind-happ-seq-leq_i-le_j*:
assumes *consec-htps* $\pi s t_i t_j$
and *wf-plan* πs
and *ind-happ-seq* $\pi s hs$
shows $\forall h \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)). \text{apply-happ } h M = M$

$\langle proof \rangle$

lemma *valid-happ-seq- M_i - M_i -precond*:

assumes $\forall h \in \text{set } hs. \text{ apply-happ } h \ M_i = M_i$

and *valid-happ-seq* M_i hs M_i

shows $\forall (t_a, as) \in \text{set } hs. \forall a \in \text{set } as. M_i \stackrel{c}{\models} \text{precondition } a$

$\langle proof \rangle$

lemma *valid-happ-seq-leq_i-le_j-aux1*:

assumes *consec-htps* πs t_i t_j

and *wf-plan* πs

and *ind-happ-seq* πs hs

and *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*le* t_j) hs)) M_j

shows $\forall (t_a, as) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs)). \forall a \in \text{set } as.$

$M_i \stackrel{c}{\models} \text{precondition } a$

$\langle proof \rangle$

lemma *valid-happ-seq-leq_i-le_j-aux2*:

assumes $\forall h \in \text{set } hs. \text{ apply-happ } h \ M_i = M_i$

and $\forall (t_a, A) \in \text{set } hs. \forall a \in \text{set } A. M_i \stackrel{c}{\models} \text{precondition } a$

and $\forall (t_a, A) \in \text{set } hs. \forall (t_a', A') \in \text{set } hs.$

$\forall a \in \text{set } A. \forall a' \in \text{set } A'. \text{ acts-non-intrf } a \ a'$

shows *valid-happ-seq* M_i hs M_i

$\langle proof \rangle$

end — Context *wf-ast-problem*

context *ast-problem*

begin

All invariants from *invs-of-plan-at* are satisfied iff an induced happening sequence is valid in between two consecutive happening time points.

lemma (**in** *wf-ast-problem*) *valid-happ-seq-leq_i-le_j-invs-of-plan-at*:

assumes *wf-plan* πs

and *consec-htps* πs t_i t_j

and *ind-happ-seq* πs hs

shows *valid-happ-seq* M (*dropWhile* (*leq* t_i) (*takeWhile* (*le* t_j) hs)) M

$\longleftrightarrow (\forall i \in \text{invs-of-plan-at } t_j \ \pi s. M \stackrel{c}{\models} i)$

$\langle proof \rangle$

lemma *drop-take-leq_i-le-leq_j-aux*:

assumes $t_k \leq t_j$

and *strict-sorted* (*map* *fst* hs)

shows *takeWhile* (*le* t_k) hs @ *takeWhile* (*leq* t_j) (*dropWhile* (*le* t_k) hs) = *takeWhile* (*leq* t_j) hs

$\langle proof \rangle$

We can split up a happening sequence via *dropWhile* and *takeWhile*

lemma *drop-take-leq_i-le-leq_j-split-app*:

assumes $t_i < t_k$ **and** $t_k \leq t_j$
and *strict-sorted* (*map fst hs*)
shows $\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_k) \text{ hs}) @ \text{dropWhile } (\text{le } t_k) (\text{takeWhile } (\text{leq } t_j) \text{ hs})$
 $= \text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) \text{ hs})$
 $\langle \text{proof} \rangle$

Some properties of *valid-happ-seq* that are needed later.

lemma *valid-happ-seq-app-iff*:
 $\text{valid-happ-seq } M_0 (\text{hs}_1 @ \text{hs}_2) M_j \longleftrightarrow (\exists M_i. \text{valid-happ-seq } M_0 \text{ hs}_1 M_i \wedge \text{valid-happ-seq } M_i \text{ hs}_2 M_j)$
 $\langle \text{proof} \rangle$

lemma *valid-happ-seq-split-iff*:
 $\text{valid-happ-seq } M_0 \text{ hs } M_j \longleftrightarrow (\exists M_i. \text{valid-happ-seq } M_0 (\text{takeWhile } f \text{ hs}) M_i \wedge \text{valid-happ-seq } M_i (\text{dropWhile } f \text{ hs}) M_j)$
 $\langle \text{proof} \rangle$

Every happening sequence contains a happening at every happening time point.

lemma *ind-happ-seq-happ-at-htps*:
assumes *is-htp* $\pi s \ t_i$
and *ind-happ-seq* $\pi s \ \text{hs}$
shows $\exists A_i. (t_i, A_i) \in \text{set } \text{hs}$
 $\langle \text{proof} \rangle$

lemma *distinct-single-set-list*: $\text{set } xs = \{x\} \implies \text{distinct } xs \implies xs = [x]$
 $\langle \text{proof} \rangle$

An induced happening sequence contain exactly one happening for every happening time point.

lemma *ind-happ-seq-le_i-leq_i*:
assumes *is-htp* $\pi s \ t_i$
and *ind-happ-seq* $\pi s \ \text{hs}$
shows $\exists A_i. \text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) \text{ hs}) = [(t_i, A_i)]$
 $\langle \text{proof} \rangle$

invs-of-plan-at is valid for a single happening time point iff any induced happening sequence is valid from the previous happening time point to the happening time point.

lemma (**in** *wf-ast-problem*) *valid-happ-seq-valid-state-seq-consec-htps*:
assumes *wf-plan* πs
and *consec-htps* $\pi s \ t_i \ t_j$
and *ind-happ-seq* $\pi s \ \text{hs}$
shows $\text{valid-happ-seq } M (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{leq } t_j) \text{ hs})) M'$
 $\longleftrightarrow \text{valid-state-seq } M [t_j] \ \pi s \ M'$
 $\langle \text{proof} \rangle$

lemma *cons-consec-https*:
assumes *https-seq* πs *https*
and $https = ts_1 @ t_i \# t_j \# ts_2$
shows *consec-https* πs t_i t_j
 $\langle proof \rangle$

lemma (in *wf-ast-problem*) *valid-happ-seq-valid-state-seq-i-to-j-aux*:
fixes πs
assumes *https-seq* πs *https*
and $https = ts_1 @ t_i \# ts_2 @ t_j \# ts_3$
and *wf-plan* πs
and *ind-happ-seq* πs *hs*
shows *valid-happ-seq* M (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) *hs*)) M'
 $\longleftrightarrow$ *valid-state-seq* M ($ts_2 @ [t_j]$) πs M'
 $\langle proof \rangle$

lemma *prod-list-Nil*: $\forall x_1 x_2. (x_1, x_2) \notin set\ xs \implies xs = []$
 $\langle proof \rangle$

An induced happening sequence is only Nil iff the original plan was Nil

lemma *ind-happ-seq-Nil*: *ind-happ-seq* πs *hs* $\implies \pi s = [] \longleftrightarrow hs = []$
 $\langle proof \rangle$

lemma *https-seq-Nil-iff*:
assumes *https-seq* πs *https*
shows $\pi s = [] \longleftrightarrow https = []$
 $\langle proof \rangle$

lemma *non-Nil-sorted-last-max*:
assumes ($xs::('a::linorder)\ list$) $\neq []$
and *sorted* *xs*
and $x_{max} = last\ xs$
shows $\forall x \in set\ xs. x \leq x_{max}$
 $\langle proof \rangle$

Proof for the existence of a sequence of happening time point for every plan.
 Needed later in the proof for the completeness of the semantics.

lemma *https-seq-exists*: $\exists https. https-seq\ \pi s\ https$
 $\langle proof \rangle$

end

context *wf-ast-problem*
begin

Two induced happening sequences contain the same ground actions in between two consecutive happening time points.

lemma *ind-happ-seqs-same-acts-leq_i-le_j*:
assumes *consec-htps* πs t_i t_j
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and $(t_a, A) \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_1))$
and $a \in \text{set } A$
shows $\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_2)) \wedge a \in \text{set } A'$
<proof>

If one induced happening sequence is a valid happening sequence in between two consecutive happening time points, then every other induced happening sequence is also valid in between those consecutive happening time point.

lemma *valid-happ-seq-leq_i-le_j*:
assumes *consec-htps* πs t_i t_j
and *wf-plan* πs
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and *valid-happ-seq* M_i $(\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_1))$ M_j
shows *valid-happ-seq* M_i $(\text{dropWhile } (\text{leq } t_i) (\text{takeWhile } (\text{le } t_j) hs_2))$ M_j
<proof>

An induced happening sequence contain exactly one happening for every happening time point.

lemma *ind-happ-seq-le_i-leq_i*:
assumes *is-htp* πs t_i
and *ind-happ-seq* πs hs
shows $\exists A_i. \text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) hs) = [(t_i, A_i)]$
<proof>

Two induced happening sequences contain the same ground actions for every happening time point.

lemma *ind-happ-seqs-same-acts-le_i-leq_i*:
assumes *is-htp* πs t_i
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and $(t_a, A) \in \text{set } (\text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) hs_1))$
and $a \in \text{set } A$
shows $\exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{le } t_i) (\text{takeWhile } (\text{leq } t_i) hs_2)) \wedge a \in \text{set } A'$
<proof>

With $\llbracket \text{is-htp } ?\pi s ?t_i; \text{ind-happ-seq } ?\pi s ?hs_1; \text{ind-happ-seq } ?\pi s ?hs_2; (?t_a, ?A) \in \text{set } (\text{dropWhile } (\text{le } ?t_i) (\text{takeWhile } (\text{leq } ?t_i) ?hs_1)); ?a \in \text{set } ?A \rrbracket \implies \exists t_a' A'. (t_a', A') \in \text{set } (\text{dropWhile } (\text{le } ?t_i) (\text{takeWhile } (\text{leq } ?t_i) ?hs_2)) \wedge ?a \in \text{set } A'$ we can prove: If we have one induced happening sequence that is a valid happening sequence for a single happening time point, then every other induced happening sequence is also valid for that happening time point.

lemma *valid-happ-seq-le_i-leq_i*:
assumes *is-htp* πs t_i
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and *valid-happ-seq* M_i (*dropWhile* (*le* t_i) (*takeWhile* (*leq* t_i) hs_1)) M_j
shows *valid-happ-seq* M_i (*dropWhile* (*le* t_i) (*takeWhile* (*leq* t_i) hs_2)) M_j
<proof>

Putting together $\llbracket wf\text{-}ast\text{-}problem\ ?P; consec\text{-}https\ ?\pi s\ ?t_i\ ?t_j; ast\text{-}problem.wf\text{-}plan\ ?P\ ?\pi s; ast\text{-}problem.ind\text{-}happ\text{-}seq\ ?P\ ?\pi s\ ?hs_1; ast\text{-}problem.ind\text{-}happ\text{-}seq\ ?P\ ?\pi s\ ?hs_2; valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (le\ ?t_i)\ (takeWhile\ (le\ ?t_j)\ ?hs_1))\ ?M_j \rrbracket \implies valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (leq\ ?t_i)\ (takeWhile\ (le\ ?t_j)\ ?hs_2))\ ?M_j$ and $\llbracket is\text{-}htp\ ?\pi s\ ?t_i; ind\text{-}happ\text{-}seq\ ?\pi s\ ?hs_1; ind\text{-}happ\text{-}seq\ ?\pi s\ ?hs_2; valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (le\ ?t_i)\ (takeWhile\ (leq\ ?t_i)\ ?hs_1))\ ?M_j \rrbracket \implies valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (le\ ?t_i)\ (takeWhile\ (leq\ ?t_i)\ ?hs_2))\ ?M_j$.
If we have one induced happening sequence that is valid from a happening time point to the next consecutive happening time point, that every other induced happening sequence is also valid for that interval.

lemma *valid-happ-seq-consec-https*:
assumes *consec-https* πs t_i t_j
and *wf-plan* πs
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs_1)) M_j
shows *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs_2)) M_j
<proof>

lemma *https-state-trace-unique-i-to-j-aux*:
fixes πs
assumes *https-seq* πs $https$
and $https = ts_1 @ t_i \# ts_2 @ t_j \# ts_3$
and *wf-plan* πs
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs_1)) M_j
shows *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs_2)) M_j
<proof>

With $\llbracket wf\text{-}ast\text{-}problem\ ?P; consec\text{-}https\ ?\pi s\ ?t_i\ ?t_j; ast\text{-}problem.wf\text{-}plan\ ?P\ ?\pi s; ast\text{-}problem.ind\text{-}happ\text{-}seq\ ?P\ ?\pi s\ ?hs_1; ast\text{-}problem.ind\text{-}happ\text{-}seq\ ?P\ ?\pi s\ ?hs_2; valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (leq\ ?t_i)\ (takeWhile\ (leq\ ?t_j)\ ?hs_1))\ ?M_j \rrbracket \implies valid\text{-}happ\text{-}seq\ ?M_i\ (dropWhile\ (leq\ ?t_i)\ (takeWhile\ (leq\ ?t_j)\ ?hs_2))\ ?M_j$ we can now proof that if one induced happening sequence is valid from a happening time point to another happening time point (not just consecutive), than every other induced happening sequence is valid for between those two happening time points.

lemma *valid-happ-seq-htp-i-to-htp-j*:
assumes $t_i < t_j$
and *is-htp* πs t_i
and *is-htp* πs t_j
and *wf-plan* πs
and *ind-happ-seq* πs hs_1
and *ind-happ-seq* πs hs_2
and *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs_1)) M_j
shows *valid-happ-seq* M_i (*dropWhile* (*leq* t_i) (*takeWhile* (*leq* t_j) hs_2)) M_j
<proof>

lemma *happ-tp-leq-t_{min}-t_{max}*:
assumes $(t_\pi, \pi) \in \text{set } \pi s$
and *inst-of-plan-action* πs (t_π, π) (t_a, a)
shows $(\exists t_j. \text{is-htp } \pi s t_j \wedge t_j \leq t_a) \wedge (\exists t_j. \text{is-htp } \pi s t_j \wedge t_a \leq t_j)$
<proof>

Proof that every happening of a induced happening sequence is in between the smallest and largest happening time point.

lemma *ind-happ-seq-htps-leq-t_{min}-t_{max}*:
fixes πs
assumes *ind-happ-seq* πs ($hs' @ hs$)
and $\forall t_i. \text{is-htp } \pi s t_i \longrightarrow t_{\min} \leq t_i$
and $\forall t_i. \text{is-htp } \pi s t_i \longrightarrow t_i \leq t_{\max}$
shows $\forall (t_a, A) \in \text{set } hs. t_{\min} \leq t_a \wedge t_a \leq t_{\max}$
<proof>

lemma *valid-happ-seq-state-unique*:
assumes *valid-happ-seq* M_0 hs M_i **and** *valid-happ-seq* M_0 hs M_i'
shows $M_i = M_i'$
<proof>

lemma *valid-happ-seq-iff-valid-state-seq*:
fixes πs
assumes *htps-seq* πs $htps$
and *wf-plan* πs
and *ind-happ-seq* πs hs
shows *valid-happ-seq* M_0 hs $M_n \longleftrightarrow \text{valid-state-seq } M_0$ $htps$ πs M_n
<proof>

lemma (in *wf-ast-problem*) *plan-happ-path-iff-valid-state-seq*:
assumes *wf-plan* πs **and** *htps-seq* πs $htps$
shows *plan-happ-path* M πs $M' \longleftrightarrow \text{valid-state-seq } M$ $htps$ πs M'
<proof>

lemmas *plan-happ-path-iff-valid-state-seq-iff'*
 $= \text{wf-ast-problem.plan-happ-path-iff-valid-state-seq}[of P, OF \text{wf-ast-problem.intro}]$

lemma *strict-sorted* $xs \implies \text{strict-sorted } ys \implies \text{set } xs = \text{set } ys \implies xs = ys$

<proof>

Proof that a sequence of happening time points is unique for every plan

lemma *https-seq-unique*:
 assumes *https-seq* πs *https*₁
 and *https-seq* πs *https*₂
 shows *https*₁ = *https*₂
<proof>

lemma *valid-plan-from2-iff*:
 assumes *wf-problem*
 shows *valid-plan-from2* $I \pi s \longleftrightarrow \text{valid-plan-from } I \pi s$
<proof>

lemma
 assumes *wf-problem*
 shows *valid-plan2* $\pi s \longleftrightarrow \text{valid-plan } \pi s$
 <proof>
 find-theorems *acts-of-plan-at*
end — Context of *ast-problem*

end — Theory

References

- [1] M. Abdulaziz and L. Koller. Formal semantics and formally verified validation for temporal planning. In *The 36th AAAI Conference on Artificial Intelligence (AAAI)*, 2022.
- [2] M. Abdulaziz and P. Lammich. AI Planning Languages Semantics. *Arch. Formal Proofs*, 2020.